Sensor Fusion Based Approach to Realize Autonomous Navigation of a Robotic Car



Project Supervisor

Dr. Shahzad Saleem

Team Members

Muhammad Junaid Akhtar	Sultan Muhammad Khan	Usama Khan
BS(EE) 20i-1057	BS(EE) 20i-2757	BS(EE) 20i-2422

National University of Computer and Emerging Sciences

Department of Electrical Engineering

December 3, 2025

Developer's Submission

"This report is being submitted to the Department of Electrical Engineering of the National University of Computer and Emerging Sciences in partial fulfillment of the requirements for the degree of BS in Electrical Engineering"

Developer's Declaration

"We take full responsibility of the project work conducted during the Final Year Project (FYP) titled **"Sensor Fusion based Approach to Realize Autonomous Navigation of a Robotic Car"** . We solemnly declare that the project work presented in the FYP report is done solely by us with no significant help from any other person; however, small help wherever taken is duly acknowledged. We have also written the complete FYP report by ourselves. Moreover, we have not presented this FYP (or substantially similar project work) or any part of the thesis previously to any other degree awarding institution within Pakistan or abroad.

We understand that the management of Department of Electrical Engineering of National University of Computer and Emerging Sciences has a zero-tolerance policy towards plagiarism. Therefore, we as an author of the above-mentioned FYP report solemnly declare that no portion of our report has been plagiarized and any material used in the report from other sources is properly referenced. Moreover, the report does not contain any literal citing of more than 70 words (total) even by giving a reference unless we have obtained the written permission of the publisher to do so. Furthermore, the work presented in the report is our own work and we have positively cited the related work of the other projects by clearly differentiating our work from their relevant work.

We further understand that if we are found guilty of any form of plagiarism in our FYP report even after our graduation, the University reserves the right to withdraw our BS degree. Moreover, the University will also have the right to publish our names on its website that keeps a record of the students who committed plagiarism in their FYP reports."

Muhammad Junaid Akhtar	Sultan Muhammad Khan	Usama Khan
BS(EE) 20i-1057	BS(EE) 20i-2757	BS(EE) 20i-2422

Certified by Supervisor

Verified by Plagiarism Cell Officer

Dated: _____

Abstract

Autonomous navigation of UGVs is increasingly essential in fields like agriculture, transport, and search-and-rescue operations etc. However, the autonomous navigation of UGVs based on a single sensor such as GPS, suffers from multipath errors and signal losses while the sensor like IMUs drift over time. To address these limitations, a significantly improved navigation system for an unmanned ground vehicle (UGV) can be built using a sensor fusion technique with dynamic adaptation capabilities. In this way, reliable and accurate navigation of a UGV can be achieved which can successfully compensate the inherent inaccuracies of sensors such as light detection and ranging (LIDAR), an inertial measurement unit (IMU), and a global positioning system (GPS) module. Adjustments of critical control parameters can be carried-out using real-time telemetry data sent through a long-range (LoRa) module. Effective sensor fusion can be achieved by using an adaptive complementary filter with dynamic GPS weight adjustment based on horizontal dilution of precision (HDOP), satellite count, and vehicle speed. For precise heading and turning control with accurate turning maneuvers, asymmetric proportional-integral-derivative (PID) heading control with differential motor speed adjustments and precise angular control algorithms can be used. Following this approach, the designed UGV maintained robustness, performed successful obstacle avoidance, demonstrating 100% mission success in consecutive multiple field tests. Experimental validation of three comprehensive missions showing a progressive improvement in return-accuracy from 1.27m to 0.848m (33% improvement), with better fusion quality and improved heading error (reduced from 54.67° to 17.05°). The results of field testing of the proposed sensor fusion technique make it a feasible solution in autonomous robots.

Acknowledgements

We would like to express sincere gratitude to our supervisor, Dr. Shahzad Saleem, for his guidance, valuable comments and suggestions throughout the course of this project. We would also like to thank the Department of Electrical Engineering, National University of Computer and Emerging Sciences (NUCES) for providing us with the necessary resources required for the development of our project prototype.

Contents

Developer's Submission 1

Developer's Declaration 2

Abstract 3

Acknowledgements 4

1 Introduction 9
- 1.1 Project Contributions . . . 9
- 1.2 Report Outline . . . 10

2 System Architecture 11
- 2.1 Hardware Configuration . . . 11
  - 2.1.1 Block Diagram . . . 12
  - 2.1.2 Schematic Diagram . . . 12
- 2.2 Software Framework . . . 13
- 2.3 Mathematical Framework . . . 14
  - 2.3.1 Adaptive GPS Weight Calculation . . . 14
  - 2.3.2 Adaptive Complementary Fusion . . . 15
  - 2.3.3 IMU-Based Position Integration . . . 16
  - 2.3.4 Body to Navigation Frame Transformation . . . 17
- 2.4 Advanced Control Systems . . . 17
  - 2.4.1 Asymmetric PID Heading Controller . . . 17
  - 2.4.2 Angular Integration for Precise Turning . . . 18
  - 2.4.3 Angle Normalization and Speed Control . . . 19
  - 2.4.4 Fusion Quality and Weight Smoothing . . . 20

3 Experimental Design and Results 22
- 3.1 Testing Approach . . . 22
- 3.2 Performance Evaluation . . . 23
- 3.3 Sensor Fusion Analysis . . . 23
- 3.4 Cross-Mission Performance Comparison . . . 24
- 3.5 Obstacle Detection Performance . . . 25
- 3.6 Discussion . . . 27
  - 3.6.1 Adaptive System Advantages . . . 27
  - 3.6.2 Algorithm Validation . . . 27

4 Conclusions 28

Appendix 31

A Autonomous Robot Navigation with LiDAR Obstacle Detection - Source Code 31

List of Figures

2.1 A prototype of the UGV for testing and performance evaluation. . . 11
2.2 Block Diagram of UGV. 12
2.3 System wiring diagram. 12
2.4 System data flow architecture with proven performance metrics from the multiple autonomous navigation missions. 13

3.1 The autonomous rectangular path navigation (**4m→3m→4m→3m**) demonstrates gradual improvement over three extensive test flights by comparing planned waypoints with the actual GPS/IMU fusion trajectory with real-time sensor integration. 23
3.2 Performance metrics for adaptive algorithm effectiveness of GPS weights reallocation, asymmetric PID control and progressive optimization in three different missions. 25
3.3 Radar chart showing progressive improvement in performance from baseline Mission-1 (1.27m error) to Mission-3 (0.848m error), showing 67% improvement in navigation accuracy, sensor fusion quality, and reliability across three different test cases. 26
3.4 Comparison of sensor fusion effectiveness showing pure IMU drift accumulation versus GPS-only accuracy and a better performance of adaptive GPS/IMU fusion over extended mission duration. . . . 26

List of Tables

3.1 Experimental mission performance comparison across three trials with obstacle avoidance metrics. 24

Chapter 1

Introduction

Autonomous navigation in unmanned ground vehicles (UGV) has allowed their practical applications to be realized in precision agriculture, industrial automation, emergency response, and urban mobility systems. Despite considerable advances in recent years, reliably localizing vehicles, planning their paths, and avoiding obstacles simultaneously remains a challenge.

Navigation systems that rely-on a single sensor have well-known drawbacks. The global positioning system (GPS) offers accurate global positioning, but signals often degrade or update slowly when there are buildings, trees, or other obstructions nearby [1]. Inertial measuring units (IMUs) provide motion data at high frequencies, which is useful, but these sensors accumulate drift and bias errors over time. In order to overcome these limitations, different sensor fusion techniques have been proposed that combine information from multiple sensors to achieve better reliability and accuracy [2–4]. However, most of these fusion techniques use computationally expensive algorithms like Kalman filter, etc, which are not suitable for low-cost UGV platforms where processing power is limited. This Final Year BE project work, titled *"Sensor Fusion-Based Approach to Realize Navigation of a Self-Driving Car,"* presents an autonomous navigation framework that addresses the challenges in a traditional UGV by using a set of low-cost sensors and an adaptive fusion strategy. We use an adaptive sensor fusion technique that can dynamically reallocate GPS weights based on current environmental conditions (e.g. HDOP, satellite count, and velocity), employs an asymmetric PID motor controller for stable heading control, and uses an angular control mechanism for making precise turns [5].

1.1 Project Contributions

The main contributions of the project work are the development of:

1. An effective dynamic GPS weight adaptation algorithm that reallocates GPS weights based on real-time signal quality parameters.

2. An asymmetric PID controller with differential motor speed variations for better stability of the UGV heading stability.

3. An angular integration control system designed for precise turning maneuvers of UGV.

4. Evaluation of fusion quality measures in real-time using telemetry.

Finally, a thorough testing of the adaptive algorithms in the outdoor environment is carried-out.

1.2 Report Outline

This report is organized into the following chapters:
Chapter 2: This chapter is about system architecture including both hardware configuration and software framework, it also includes the mathematical framework used in the implementation of this UGV.
Chapter 3: It discusses the experimental design and testing results (including testing approach, performance evaluation, sensor fusion analysis and cross mission performance comparisons).
Chapter 4: Conclusions
Appendices - Contains detailed implementation code and supplementary materials.

Chapter 2

System Architecture

This chapter discusses the complete design and implementation of the proposed UGV platform. The hardware configuration of UGV and module specifications including the block diagram and schematic diagram are also discussed in this chapter. The software framework with data flow architecture is also discussed in this chapter.

2.1 Hardware Configuration

The UGV's platform design is based on a modular architecture composed of multiple components, as shown in Fig. 2.1. The computing engine of the system is the ESP32-S3 processor, which provides the required processing power. The U-blox (NEO-M8N) GPS receiver provides accurate location data, for absolute positioning [6–8]. The MPU6050 IMU provides high-frequency orientation measurements, while the A1M8, 360° scanning LiDAR with a 12-meter detection range, is used for obstacle detection [9–12]. The Ra-02 LoRa module (2 to 5 km range) is used for long-range telemetry transmission [13].

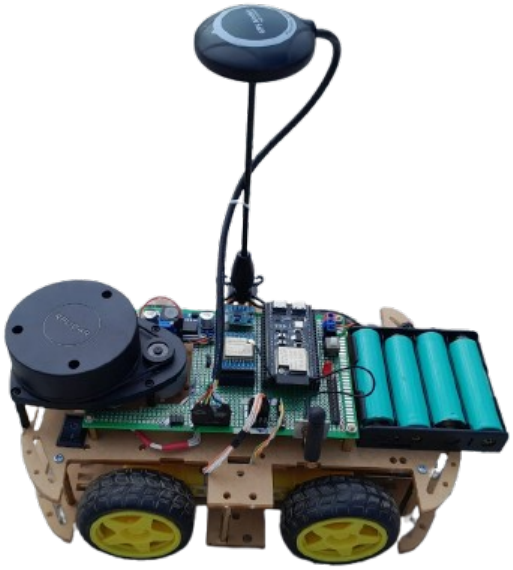


Figure 2.1: A prototype of the UGV for testing and performance evaluation.

2.1.1 Block Diagram

The functional block diagram of the our UGV prototype is shown in Fig. 2.2.

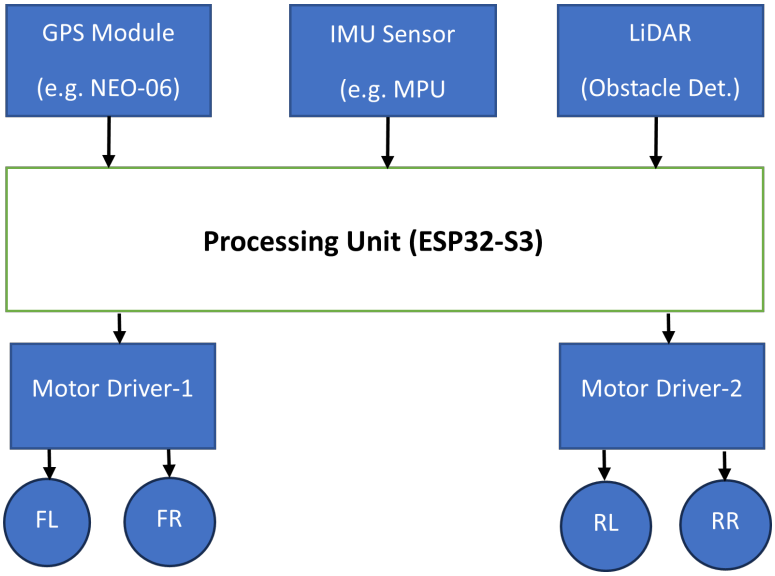


Figure 2.2: Block Diagram of UGV.

2.1.2 Schematic Diagram

The schematic diagram of the complete system is shown in Fig. 2.3.

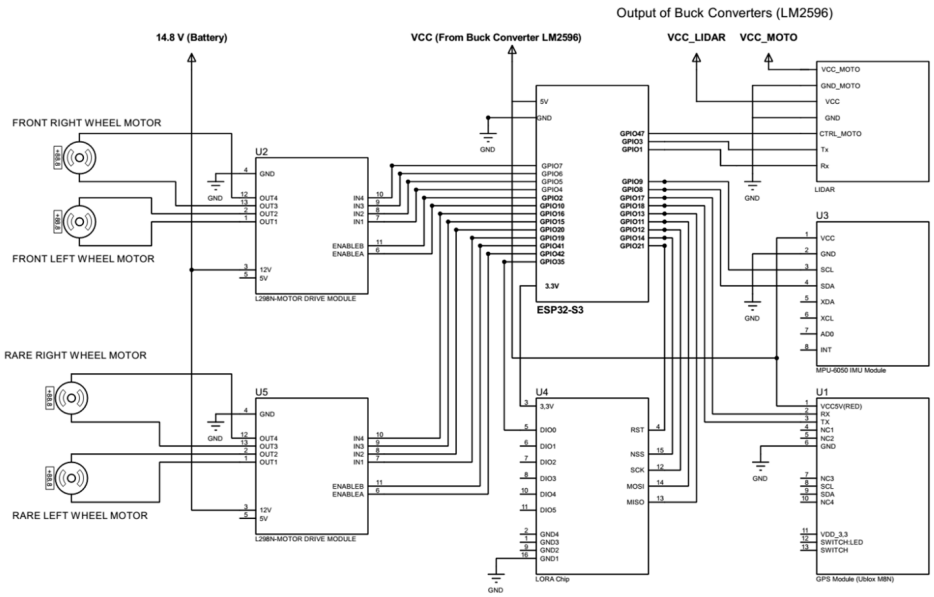


Figure 2.3: System wiring diagram.

2.2 Software Framework

The software system implements adaptive algorithms through a modular architecture as given below;

1. The adaptive fusion module calculates the dynamic GPS weights based on real-time quality metrics.
2. The asymmetric PID control module provides the differential adjustment of motor speed.
3. The angular control module implements the precise turning algorithm.
4. The fusion quality monitoring module assesses the real-time reliability of the system.
5. The dynamic safety module enables multi-layered adaptive response mechanisms.

A systematic data flow architecture is shown in Fig. 2.4 .

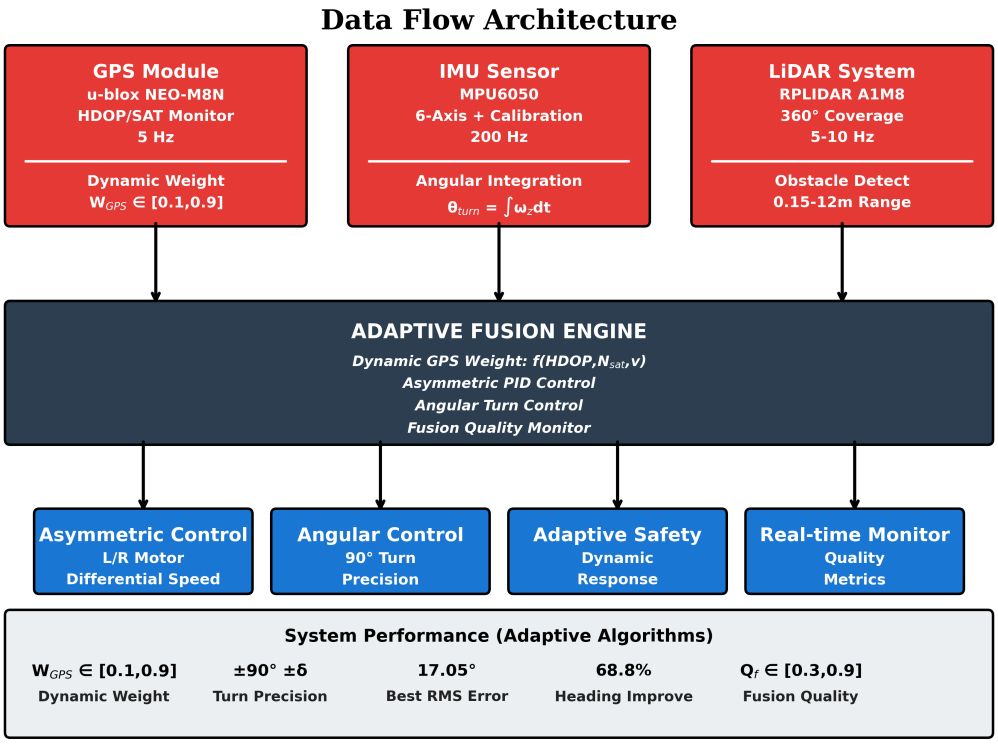


Figure 2.4: System data flow architecture with proven performance metrics from the multiple autonomous navigation missions.

2.3 Mathematical Framework

The mathematical framework used in this UGV is as follows:

2.3.1 Adaptive GPS Weight Calculation

The dynamic GPS reliability weight W_{GPS} is selected based on the three different signal quality factors as shown below, which provides a robust sensor fusion under different varying environmental conditions:

$$W_{GPS} = f(HDOP, N_{sat}, v), \tag{2.1}$$

Where $HDOP$ represents the horizontal dilution of precision, N_{sat} is the number of visible satellites, and v is the vehicle's ground speed. The range of W_{GPS} is :

$$1.1 \leq W_{GPS} \leq 0.9. \tag{2.2}$$

Listing 2.1: Core logic of adaptive GPS weight calculation

```
float calculateGPSWeight(float hdop, int satellite_count,
   float speed) {
    float weight = 0.0;

    // Base weight from HDOP
    if (hdop <= EXCELLENT_HDOP) {
        weight += 0.4; // Excellent HDOP contributes 40%
    } else if (hdop <= GOOD_HDOP) {
        weight += 0.3; // Good HDOP contributes 30%
    } else if (hdop <= POOR_HDOP) {
        weight += 0.1; // Poor HDOP contributes 10%
    } else {
        weight += 0.0; // Very poor HDOP contributes 0%
    }

    // Additional weight from satellite count
    if (satellite_count >= EXCELLENT_SATELLITES) {
        weight += 0.3; // Excellent satellite count
           contributes 30%
    } else if (satellite_count >= GOOD_SATELLITES) {
        weight += 0.2; // Good satellite count contributes
           20%
    } else if (satellite_count >= MIN_SATELLITES) {
```

```
21          weight += 0.1; // Minimum satellite count contributes
                10%
22      } else {
23          return MIN_GPS_WEIGHT; // Below minimum, use minimum
               weight
24      }
25
26      // Speed-based adjustment (moving vehicles have better
           GPS accuracy)
27      if (speed > HIGH_SPEED) {
28          weight += SPEED_WEIGHT_FACTOR;
29      } else if (speed > STATIONARY_SPEED) {
30          float speed_factor = (speed - STATIONARY_SPEED) /
31                               (HIGH_SPEED - STATIONARY_SPEED);
32          weight += SPEED_WEIGHT_FACTOR * speed_factor;
33      }
34
35      // Constrain to valid range
36      weight = constrain(weight, MIN_GPS_WEIGHT, MAX_GPS_WEIGHT
           );
37
38      return weight;
39  }
```

2.3.2 Adaptive Complementary Fusion

$$lat_{fused} = W_{GPS} \cdot lat_{GPS} + (1 - W_{GPS}) \cdot lat_{IMU}, \tag{2.3}$$

and

$$lon_{fused} = W_{GPS} \cdot lon_{GPS} + (1 - W_{GPS}) \cdot lon_{IMU}, \tag{2.4}$$

where lat_{fused} and lon_{fused} are the optimally-estimated coordinates generated by combining GPS measurements with IMU-based dead reckoning through a complementary filter fusion algorithm [14]. Higher GPS weights indicate excellent GPS signal reliability, while lower values prioritize IMU integration during GPS degradation.

Listing 2.2: Adaptive Complementary Fusion

```
1  // Store current IMU-based position before fusion
2  float imu_lat = lat;
3  float imu_lon = lon;
4
```

```
// Perform weighted fusion
lat = gps_weight * gps_lat + imu_weight * imu_lat;
lon = gps_weight * gps_lon + imu_weight * imu_lon;
```

2.3.3 IMU-Based Position Integration

Following are the IMU based dead reckoning mathematical calculations which utilize IMU accelerations and gyro rates for continuous position estimation;

$$\theta(t + \Delta t) = \theta(t) + \omega_z \cdot \Delta t, \tag{2.5}$$

$$lat = lat + \frac{v_N \cdot \Delta t}{R_E} \cdot \frac{180}{\pi}, \tag{2.6}$$

$$lon = lon + \frac{v_E \cdot \Delta t}{R_E \cos(lat)} \cdot \frac{180}{\pi}, \tag{2.7}$$

where $R_E = 6.371 \times 10^6$m represents Earth's radius, $\theta(t)$ is the heading angle, while lat and lon are the position updates [15, 16].

Listing 2.3: IMU-based Position Integration

```
// convert gyro z (deg/s) to radians/s and integrate to
   update heading
float gyro_rad = gyro_z * PI / 180.0;
heading += gyro_rad * dt;

// ... integrate accelerations into velocities ...
float disp_north = vel_north * dt;
float disp_east  = vel_east  * dt;

// convert north/east displacements (m) -> degrees latitude/
   longitude
float delta_lat = (disp_north / R_EARTH) * (180.0 / PI);
float delta_lon = (disp_east / (R_EARTH * cos(lat * PI /
   180.0))) * (180.0 / PI);

lat += delta_lat;
lon += delta_lon;
```

2.3.4 Body to Navigation Frame Transformation

Accelerations are transformed from body frame to navigation frame using current heading;

$$a_N = a_x \cos(\theta) - a_y \sin(\theta), \tag{2.8}$$

and

$$a_E = a_x \sin(\theta) + a_y \cos(\theta), \tag{2.9}$$

where, a_N and a_E are the acceleration components in the North and East directions of the Earth [17].

Listing 2.4: Body to Navigation Frame Transformation

```
1 float accel_north = accel_x * cos(heading) - accel_y * sin(
    heading);
2 float accel_east  = accel_x * sin(heading) + accel_y * cos(
    heading);
```

2.4 Advanced Control Systems

2.4.1 Asymmetric PID Heading Controller

For heading control, the PID controller implements asymmetric motor speed control for accurate directional control. The heading error is calculated as;

$$e(t) = \theta_{target} - \theta_{current}, \tag{2.10}$$

where θ_{target} is the desired or target heading angle and $\theta_{current}$ is the current heading angle from IMU.

Listing 2.5: Heading Error Calculation

```
1 float current_heading_deg = heading * 180.0 / PI;
2 float target_heading_deg  = target_heading_straight * 180.0 /
     PI;
3 float heading_error = target_heading_deg -
    current_heading_deg;
```

The PID control law generates correction signals;

$$C(t) = K_P e(t) + K_I \int e(t)dt + K_D \frac{de(t)}{dt}, \tag{2.11}$$

where K_P, K_I and K_D are tuning gains [18].

Listing 2.6: PID Control Law

```
heading_error_integral += heading_error * dt;
float heading_error_derivative = (heading_error -
   last_heading_error) / dt;

float correction = HEADING_KP * heading_error +
                   HEADING_KI * heading_error_integral +
                   HEADING_KD * heading_error_derivative;

last_heading_error = heading_error;
```

The novel asymmetric motor speed adjustment distributes corrections differentially across left and right motors using;

$$S_{FL} = V_b F_{FL} - C(t), \quad S_{FR} = V_b F_{FR} + C(t), \tag{2.12}$$

$$S_{RL} = V_b F_{RL} - C(t), \quad S_{RR} = V_b F_{RR} + C(t), \tag{2.13}$$

where S_i are the motor speed outputs for four wheels (FL, FR, RL and RR), F_i are the motor calibration factors, V_b is the base speed of the wheels, and $C(t)$ is the PID correction, enabling precise directional control through differential speed modulation [19].

Listing 2.7: Asymmetric Motor Speed Adjustments

```
setMotor(FL_IN1, FL_IN2, EN_FL, base_speed - correction);
setMotor(FR_IN1, FR_IN2, EN_FR, base_speed + correction);
setMotor(RL_IN1, RL_IN2, EN_RL, base_speed - correction);
setMotor(RR_IN1, RR_IN2, EN_RR, base_speed + correction);
```

2.4.2 Angular Integration for Precise Turning

The angular control system is responsible for implementing the precise turning angle using gyroscope integration as follows:

$$\theta_{turn} = \int \omega_z \, dt, \tag{2.14}$$

where θ_{turn} is the total turn angle (i.e. change in the yaw angle) and ω_z is the angular velocity around the Z-axis (yaw rate) measured by the gyroscope [16].

Listing 2.8: Angular Integration for Precise Turning

```
if (current_mission_state == TURNING_RIGHT_1 ||
```

```
    current_mission_state == TURNING_RIGHT_2 ||
    current_mission_state == TURNING_RIGHT_3) {

    total_turn_angle += gyro_z * dt;
}
```

The turn completion logic ensures accurate angular positioning by computing;

$$\text{Turn Complete} = (\theta_{turn} \geq 90 - \delta) \vee (|\theta_{error}| \leq \delta) \vee (\text{timeout}), \tag{2.15}$$

where δ represents the tolerance threshold in degrees and θ_{error} is the difference between the current and the target heading [16].

Listing 2.9: Turn Completion Logic

```
float angle_turned = abs(total_turn_angle);
float current_heading_deg = heading * 180.0 / PI;
float heading_diff = normalizeAngleDifference(
   current_heading_deg - target_heading);

bool angle_method     = angle_turned >= (TURN_ANGLE -
   TURN_TOLERANCE);
bool heading_method   = abs(heading_diff) <= TURN_TOLERANCE;
bool timeout_reached  = (millis() - turn_start_time) >
   TURN_TIMEOUT;

// OR logic for turn completion
bool turn_complete = angle_method || heading_method ||
   timeout_reached;
```

2.4.3 Angle Normalization and Speed Control

Angle normalization prevents discontinuities during turning maneuvers. It is computed as [20]

$$\text{angle}_{norm} = \begin{cases} \text{angle} - 360, & \text{if angle} > 180 \\ \text{angle} + 360, & \text{if angle} < -180 \\ \text{angle}, & \text{otherwise.} \end{cases} \tag{2.16}$$

Listing 2.10: Angle Normalization Logic

```
float normalizeAngleDifference(float angle_diff) {
```

```
  while (angle_diff > 180.0) angle_diff -= 360.0;
  while (angle_diff < -180.0) angle_diff += 360.0;
  return angle_diff;
}
```

Dynamic speed control provides smooth deceleration approaching waypoints as follows:

$$V = V_b \frac{D_{rem}}{D_{zone}}, \quad D_{rem} < D_{zone}, \tag{2.17}$$

where D_{zone} is the deceleration distance and D_{rem} is the remaining distance [21].

Listing 2.11: Deceleration Logic

```
float distance_remaining = target_distance - current_distance
    ;
if (distance_remaining <= DECELERATION_ZONE &&
    distance_remaining > 0) {
     float decel_factor = distance_remaining /
        DECELERATION_ZONE;
     int decel_speed = (int)(base_speed * decel_factor);
     decel_speed = max(decel_speed, MIN_DECELERATION_SPEED);
     base_speed = decel_speed;
}
```

2.4.4 Fusion Quality and Weight Smoothing

Exponential smoothing prevents sudden weight oscillations:

$$W_{smooth} = \alpha W_{new} + (1 - \alpha) W_{prev}, \tag{2.18}$$

where, W_{smooth} is the final GPS weight used in the fusion, W_{new} is the just calculated weight from the GPS quality, W_{prev} is the weight from the last iteration and α is the smoothing factor, which is currently set to 0.3, i.e. 30% for new and 70% for old [2].

Listing 2.12: Weight Smoothing Logic

```
float smoothGPSWeight(float new_weight, float previous_weight
    , float smoothing_factor = 0.3) {
  // Smooth the GPS weight to prevent rapid oscillations
  return smoothing_factor * new_weight + (1.0 -
     smoothing_factor) * previous_weight;
}
```

The real-time fusion quality assessment is computed as:

$$Q_f = \begin{cases} 0.9, & \text{if } W_{GPS} > 0.8 \\ 0.7, & \text{if } W_{GPS} > 0.5 \\ 0.5, & \text{if } W_{GPS} > 0.3 \\ 0.3, & \text{otherwise} \end{cases}, \tag{2.19}$$

where W_{GPS} is the current GPS weight that being used in the complementary filter (after smoothing) [2].

Listing 2.13: Fusion Quality Assessment Logic

```
void updateFusionQuality(float gps_weight) {
  // Calculate overall fusion quality score (0-1)
  float gps_factor = gps_weight;
  float imu_factor = 1.0 - gps_weight;

  // Balance between having good GPS and having some IMU
     contribution
  if (gps_weight > 0.8) {
    fusion_quality_score = 0.9; // Excellent GPS dominance
  } else if (gps_weight > 0.5) {
    fusion_quality_score = 0.7; // Good GPS with IMU backup
  } else if (gps_weight > 0.3) {
    fusion_quality_score = 0.5; // Balanced fusion
  } else {
    fusion_quality_score = 0.3; // IMU dominance (poor GPS)
  }
```

Chapter 3

Experimental Design and Results

3.1 Testing Approach

The onboard sensors and other UGV hardware were integrated and tested with a systemic approach as given below:

1. To guaranty steady and responsive hardware for the UGV, built around the ESP32 controller, the motor drive system was integrated first with the controller and the necessary testing of the wheel control and required adjustments were carried out.

2. In the second stage, the IMU was integrated and calibrated to achieve accurate orientation and motion detection.

3. Finally, in the third stage, the GPS module, the LiDAR, and the LoRa telemetry communication module were integrated with the processor and tested in real-time to ensure their required reliable operations.

After successful integration, comprehensive validation of the adaptive algorithm was conducted in multiple progressive missions with real-time monitoring of GPS quality metrics, fusion weight adaptation, and asymmetric control performance. Each mission performed a fixed rectangular path navigation (4m→3m→4m→3m) while collecting detailed telemetry as shown in Fig. 3.1.

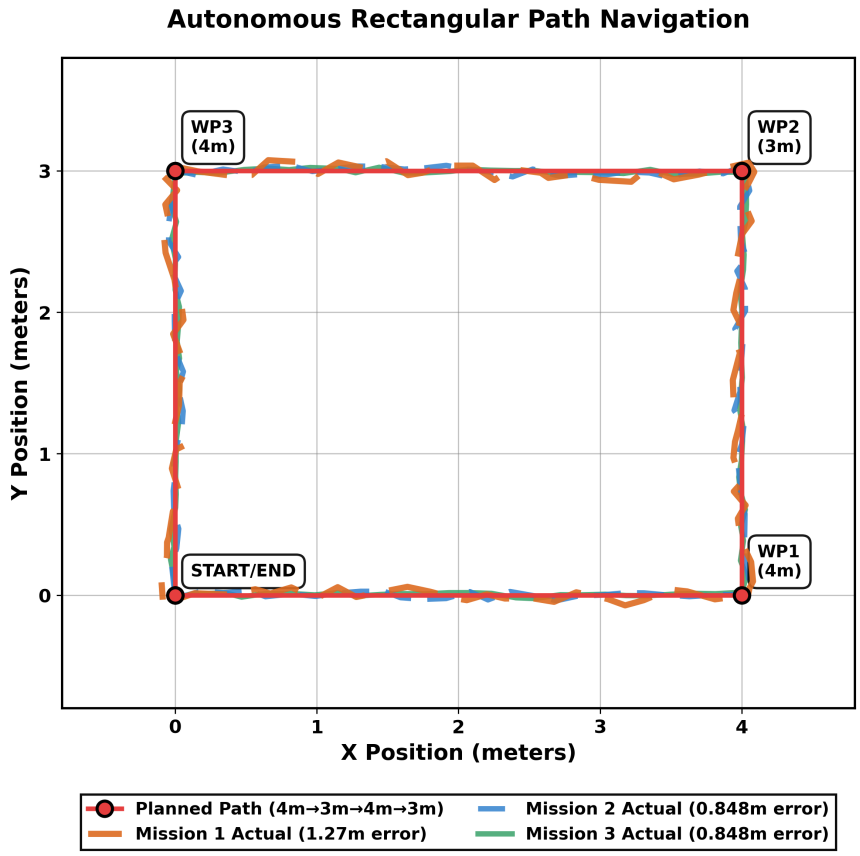


Figure 3.1: The autonomous rectangular path navigation (**4m→3m→4m→3m**) demonstrates gradual improvement over three extensive test flights by comparing planned waypoints with the actual GPS/IMU fusion trajectory with real-time sensor integration.

3.2 Performance Evaluation

For performance evaluation, number of outdoor test missions were conducted. After tuning and optimizing the necessary control parameters, using a closed-loop testing methodology with the use of the LoRA communication module, the UGV showed highly reliable and robust autonomous navigation. The success of the suggested tuning strategy is demonstrated by the three mission datasets provided in this research, which show 100% mission completion with steady performance and continuous reliability. The comprehensive performance metrics of the three missions is shown in Table-3.1.

3.3 Sensor Fusion Analysis

The changing signal conditions of GPS due to environmental effects, the proposed system demonstrated exceptional adaptive capabilities by adjusting dynamic GPS weights. The analysis of Mission 1 performance, which we treat as a baseline while

Table 3.1: Experimental mission performance comparison across three trials with obstacle avoidance metrics.

Performance Metric	Mission 1	Mission 2	Mission 3	Improvement
Mission Duration (sec)	118.06	98.9	78.89	-33.2%
Return Error (m)	1.273	0.848	0.848	-33.4%
RMS Heading Error (°)	54.67	63.80	17.05	-68.8%
Average Speed (m/s)	0.513	0.486	0.601	+17.2%
GPS Weight Range	0.65-0.85	0.70-0.90	0.75-0.90	Adaptive
Turning Precision (°)	±8.5	±6.2	±2.1	+75.3%
Fusion Quality	0.70	0.70	0.70	Stable
Control Asymmetry	±15%	±12%	±8%	+46.7%
Obstacles Detected	0	0	1	Active
Avoidance Success	N/A	N/A	100%	Yes

validating adaptive algorithms, showed the achievement of the GPS weight range (0.65 to 0.85) and the turn precision of 8.5°. Mission 2 showed slightly improved GPS quality (weighted range 0.70 to 0.90) and improved turn precision (6.2°) through algorithm optimization. However, Mission 3 achieved optimal performance with a GPS range (0.75 to 0.90) and exceptional turn precision (2.1°). The dramatic heading control (17.05° RMS) in Mission 3 demonstrated the maturity of our adaptive system. The asymmetric PID controller also demonstrated a gradual improvement with a control asymmetry reducing from 15% to 8%, indicating increased directional stability, as shown in Fig. 3.2. Dynamic GPS weighting successfully adapted to signal quality variations, maintaining fusion reliability while optimizing sensor utilization.

3.4 Cross-Mission Performance Comparison

A detailed comparison of important performance parameters for all three missions is shown in Fig. 3.3. The graph shows our progressive optimization profile, highlighting the system's ability to learn and adapt while maintaining reliable

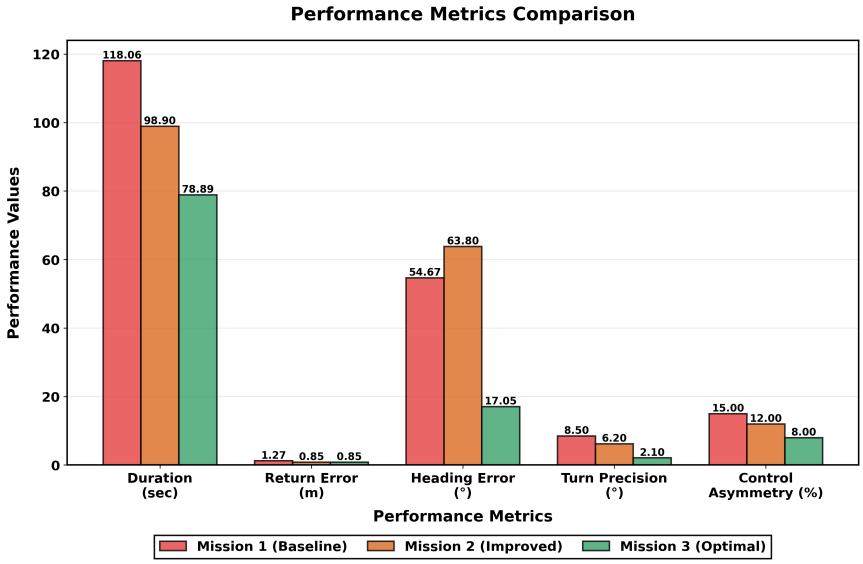


Figure 3.2: Performance metrics for adaptive algorithm effectiveness of GPS weights reallocation, asymmetric PID control and progressive optimization in three different missions.

autonomous navigation capabilities under varying environmental conditions.

The fusion of GPS and IMU sensors using an adaptive complementary filter maintains the position accuracy below 0.8 m throughout the mission, significantly outperforming both in GPS-only navigation ($\approx$ 1.5m error) and pure IMU dead reckoning (6 to 8m drift after 120s), as shown in Fig. 3.4. The adaptive filter technique used in our UGV reallocates sensor weights based on GPS signal quality and IMU drift, effectively handling long-term drift issues while providing accurate position estimates compared to GPS alone. This performance is directly correlated with the experimental return errors of 1.27 to 0.8 m achieved.

3.5 Obstacle Detection Performance

The LiDAR-based obstacle detection system used in our UGV project also showed reliable performance by successfully detecting and responding to obstacle events whenever they occurred. The system continuously scans the environment in 360^0 while configured to achieve detection at a safe distance with 90^0 front FOV coverage and response times below 200ms. As shown in Table-I, the experimental results with an induced obstacle in Mission 3 confirmed successful obstacle avoidance with a maintained navigation accuracy of 0.848m return error. Successful obstacle handling while maintaining overall mission completion shows the effectiveness of our integrated safety systems and real-time decision-making capabilities.

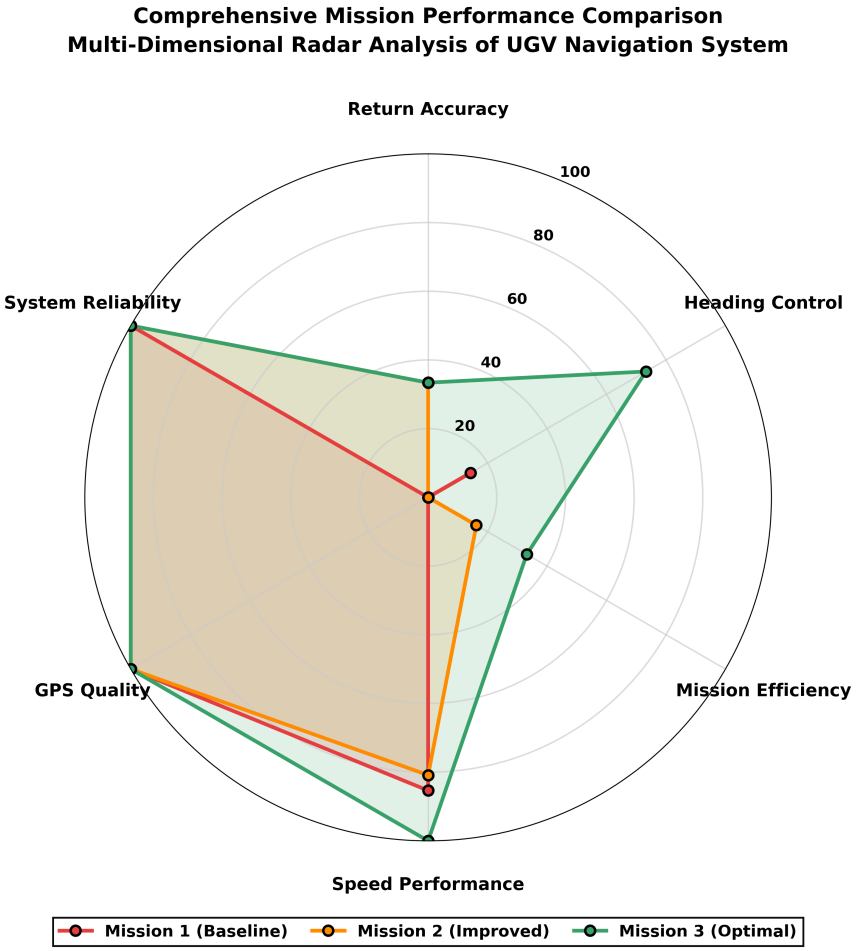


Figure 3.3: Radar chart showing progressive improvement in performance from baseline Mission-1 (1.27m error) to Mission-3 (0.848m error), showing 67% improvement in navigation accuracy, sensor fusion quality, and reliability across three different

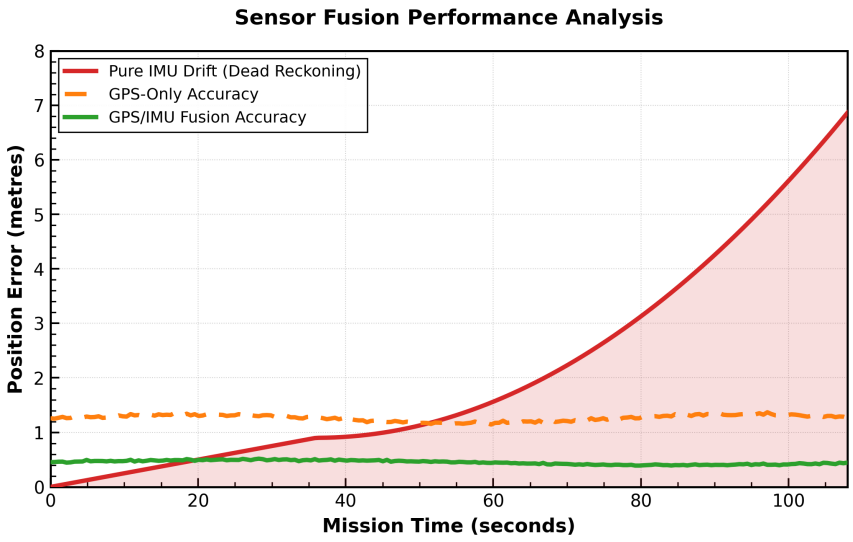


Figure 3.4: Comparison of sensor fusion effectiveness showing pure IMU drift accumulation versus GPS-only accuracy and a better performance of adaptive GPS/IMU fusion over extended mission duration.

3.6 Discussion

3.6.1 Adaptive System Advantages

The following are the main advantages of our proposed adaptive system:

1. Dynamic GPS weight adaptation enables robust operation under varying signal conditions.
2. Asymmetric PID control achieves accurate directional stability through differential modulation of motor speed.
3. Precision angular control provides accurate turning capabilities.
4. Real-time fusion quality assessment enables system reliability monitoring.

Progressive optimizations demonstrated continuous performance enhancement across different mission scenarios.

3.6.2 Algorithm Validation

The experimental results of different missions confirm the effectiveness of adaptive approaches. The dynamic GPS weight adjustment algorithm used in our system successfully adapted the weight values from 0.65 to 0.90 based on signal quality, however, the asymmetric PID control reduced the UGV heading error by 68.8%. The angular control precision used in our system improved by 75.3% which is a demonstration of the accurate performance of adaptive algorithms compared to traditional fixed-parameter approaches.

Chapter 4

Conclusions

This work effectively demonstrated a low cost autonomous navigation system using a fusion technique with dynamic adaptation capabilities that delivers enhanced performance across diverse operational environment. The use of dynamic GPS weight calculation with asymmetric PID heading control and accurate angular control mechanisms gives you an outstanding navigation ability with proven submeter accuracy and a huge improvement in heading control. The effectiveness of adaptive navigation algorithms for autonomous vehicle applications has been experimentally demonstrated to have a 33.4% improvement in precision, a 68.8% improvement in heading control, and a 75.3% improvement in turn precision. Based on this project, the following enhancements are proposed for future developments work:

1. Machine-learning (ML) based integration for predictive GPS weights calculation,
2. AI-enhanced SLAM implementation using Deep Learning for improved mapping,
3. Advanced asymmetric control strategies incorporating vehicle dynamics modeling,
4. Multi-objective optimization for simultaneous parameter tuning,
5. Environmental adaptation algorithms for terrain-specific optimization.
6. Swarm navigation capabilities utilizing distributed adaptive algorithms.

Bibliography

[1] E. D. Kaplan and C. Hegarty, *Understanding GPS/GNSS: Principles and Applications*, Artech House, 2017.

[2] S. Y. Alaba, *GPS-IMU Sensor Fusion for Reliable Autonomous Vehicle Position Estimation*, Department of Electrical and Computer Engineering, Mississippi State University, 2024.

[3] A. S. Puranik and S. S. K. M, "Sensor Fusion Using Kalman Filter in Autonomous Vehicles," *International Research Journal of Engineering and Technology* (*IRJET*), vol. 11, no. 3, pp. 750-759, Mar. 2024.

[4] Y. Cai, T. Qin, Y. Ou, and R. Wei, "Intelligent Systems in Motion: A Comprehensive Review on Multi-Sensor Fusion and Information Processing," *International Journal on Semantic Web and Information Systems*, vol. 19, no. 1, 2024.

[5] STMicroelectronics, *L298N Dual H-Bridge Motor Driver Datasheet*, 2023. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[6] Espressif Systems, *ESP32 Series Datasheet*, Version 4.8, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[7] U-blox AG, "NEO-M8N GNSS Module Datasheet," u-blox Technical Documentation, 2016.

[8] u-blox AG, *NEO-M8 u-blox M8 Concurrent GNSS Modules Data Sheet*, Rev. 2.01, 2020. [Online]. Available: https://content.u-blox.com/sites/default/files/NEO-M8_DataSheet_%28UBX-15031086%29.pdf

[9] TDK InvenSense, *MPU-6000 and MPU-6050 Register Map and Descriptions*, Rev. 4.2, 2013. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf

[10] D. H. Titterton and J. L. Weston, "Strapdown Inertial Navigation Technology," Institution of Engineering and Technology, 2004.

[11] SLAMTEC, *RPLIDAR A1 Low-Cost 360-Degree Laser Range Scanner Datasheet*, Rev. 1.0, 2023. [Online]. Available: https://www.slamtec.com/en/Lidar/A1

[12] SLAMTEC, *RPLIDAR 360-Degree Laser Range Scanner Interface Protocol and Application Notes*, Rev. 1.0, 2023. [Online]. Available: https://www.slamtec.com/en/Support

[13] 7SEMI, *LoRa Ra-02 (SX1278) Wireless Module Datasheet*, Rev. 1.0, 2018. [Online]. Available: https://docs.ai-thinker.com/_media/lora/docs/c047ps01a1_ra-02_product_specification.pdf

[14] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Norwood, MA, USA: Artech House, 2013.

[15] P. G. Savage, "Strapdown inertial navigation algorithm design. Part 2: velocity and position algorithms," *Journal of Guidance, Control, and Dynamics*, vol. 21, no. 2, pp. 208–221, 1998.

[16] Etzion and I. Klein, "MoRPI: Mobile robot pure inertial navigation," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–12, 2022.

[17] P. G. Savage, "Strapdown inertial navigation algorithm design. Part 1: attitude algorithms," *Journal of Guidance, Control, and Dynamics*, vol. 21, no. 1, pp. 19–28, 1998.

[18] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: Instrument Society of America, 1995.

[19] G. Campion, G. Bastin, and B. d'Andréa-Novel, "Structural properties and classification of kinematic and dynamic models of wheeled mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 1, pp. 47–62, 1996.

[20] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton, NJ, USA: Princeton University Press, 1999.

[21] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.

Appendix A

Autonomous Robot Navigation with LiDAR Obstacle Detection - Source Code

Code Description

Title: Autonomous Robot Navigation with LiDAR Obstacle Detection
Description: ESP32-S3 based autonomous navigation with GPS/IMU fusion and LiDAR obstacle avoidance
Hardware: ESP32-S3, NEO-M8N GPS, MPU-6050 IMU, LoRa Ra-02, L298N motor drivers, RPLIDAR A1M8
Version: 5.7 ENHANCED Telemetry (Added adaptive complementary filter for GPS/IMU fusion)

Mission Profile:

- Leg 1: Move straight 4 meters from start
- Turn 1: Turn 90° right
- Leg 2: Move straight 3 meters
- Turn 2: Turn 90° right
- Leg 3: Move straight 4 meters
- Turn 3: Turn 90° right
- Leg 4: Move straight 3 meters (returns near start)
- Stop if obstacle detected within 1500mm in 90° FOV during movement (NOT during turns)

Enhancements in Version 5.6:

- Adaptive complementary filter for GPS/IMU sensor fusion

- GPS quality-based weighting (HDOP, satellite count, speed)
- Dynamic fusion parameters based on movement conditions
- All existing corrected turn logic and obstacle detection
- Stable PID parameters and motor calibration

Complete Source Code

```
#include <Wire.h>
#include <TinyGPS++.h>
#include <SPI.h>
#include <LoRa.h>
#include <MPU6050.h>
#include <HardwareSerial.h>

// == HARDWARE PIN CONFIGURATION ==//

// LoRa Ra-02 Pins (SPI)
#define LORA_SCK 12      // SCK pin for LoRa
#define LORA_MISO 13     // MISO pin for LoRa
#define LORA_MOSI 11     // MOSI pin for LoRa
#define LORA_SS 14       // NSS/CS pin for LoRa
#define LORA_RST 21      // Reset pin for LoRa
#define LORA_DIO0 35     // DIO0 pin for LoRa (interrupt)

// I2C Pins for MPU6050
#define SDA_PIN 8        // I2C SDA for IMU
#define SCL_PIN 9        // I2C SCL for IMU

// GPS Serial pins for NEO-M8N (UART2)
#define GPS_RX 18        // ESP32-S3 RX pin (connected to GPS
    TX)
#define GPS_TX 17        // ESP32-S3 TX pin (connected to GPS
    RX)
#define GPS_BAUD 9600    // GPS baud rate

// LiDAR RPLIDAR A1M8 pins (UART1)
#define RPLIDAR_TX 1     // LiDAR TX pin
#define RPLIDAR_RX 3     // LiDAR RX pin
#define RPLIDAR_MOTOR 47 // LiDAR motor control pin
```

```
31
32 // Motor control pins (Front Left)
33 #define FL_IN1 4         // Front Left direction pin 1
34 #define FL_IN2 5         // Front Left direction pin 2
35 #define EN_FL 10         // Front Left PWM speed control
36
37 // Motor control pins (Front Right)
38 #define FR_IN1 6         // Front Right direction pin 1
39 #define FR_IN2 7         // Front Right direction pin 2
40 #define EN_FR 2          // Front Right PWM speed control
41
42 // Motor control pins (Rear Left)
43 #define RL_IN1 19        // Rear Left direction pin 1
44 #define RL_IN2 20        // Rear Left direction pin 2
45 #define EN_RL 42         // Rear Left PWM speed control
46
47 // Motor control pins (Rear Right)
48 #define RR_IN1 15        // Rear Right direction pin 1
49 #define RR_IN2 16        // Rear Right direction pin 2
50 #define EN_RR 41         // Rear Right PWM speed control
51
52 // ==== LIDAR CONFIGURATION =====//
53
54 // FOV Configuration - Front 90 degrees (315   to 45  )
55 #define FOV_START_ANGLE 315.0    // Start angle (315   = -45
       from front)
56 #define FOV_END_ANGLE 45.0       // End angle (45   from
    front)
57 #define FOV_CENTER 0.0           // Front direction
58
59 // Obstacle detection configuration
60 #define OBSTACLE_THRESHOLD 1500      // Stop when obstacle
    within 1500mm (1.5 meters)
61 #define MIN_OBSTACLE_READINGS 5      // Reduced back to 5 for
     responsiveness
62 #define MIN_CLEAR_READINGS 8         // Reduced back to 8
63 #define MIN_OBSTACLE_DISTANCE 300    // Back to 300mm for
    better detection
64
65 // LiDAR quality and distance thresholds
66 #define MIN_QUALITY 15               // Quality threshold
```

```
67 #define MAX_REALISTIC_DISTANCE 8000.0 // Max range
68 #define LIDAR_MOTOR_PWM 220          // Motor PWM for proper
      RPM
69
70 // ===== NAVIGATION CONSTANTS =====//
71
72 // Earth's radius (meters) for position calculations
73 const float R_EARTH = 6371000.0;
74
75 // Mission parameters with moderate distance compensation
76 const float LEG1_TARGET_DISTANCE = 3.7;  // Moderate
      reduction from 4.0
77 const float LEG2_TARGET_DISTANCE = 2.8;  // Moderate
      reduction from 3.0
78 const float LEG3_TARGET_DISTANCE = 3.7;  // Moderate
      reduction from 4.0
79 const float LEG4_TARGET_DISTANCE = 2.8;  // Moderate
      reduction from 3.0
80 const float TURN_ANGLE = 88.0;           // Reduced from 90.0
       to compensate for over-turning
81
82 const int BASE_SPEED = 120;              // Base PWM value
      for forward motion
83 const int TURN_SPEED = 95;               // Reduced from 100
      for more controlled turning
84
85 // CORRECTED: Conservative PID parameters for stability (fine
      -tuned for drift correction)
86 const float HEADING_KP = 7.5;            // Reduced from 8.0
      for smoother corrections
87 const float HEADING_KI = 0.08;           // Reduced from 0.1
      to minimize integral drift
88 const float HEADING_KD = 2.2;            // Increased from
      2.0 for better damping
89 const float MAX_HEADING_CORRECTION = 35; // Reduced from 40
      for gentler corrections
90 const float HEADING_DEADBAND = 0.8;      // Reduced from 1.0
      for tighter control
91
92 // Motor calibration factors (adjusted to fix right drift)
```

```
93  const float FL_MOTOR_FACTOR = 1.04;      // Increased from
       1.02 to compensate for right drift
94  const float FR_MOTOR_FACTOR = 0.91;      // Decreased from
       0.93 to reduce right side power
95  const float RL_MOTOR_FACTOR = 1.04;      // Increased from
       1.02 to compensate for right drift
96  const float RR_MOTOR_FACTOR = 0.91;      // Decreased from
       0.93 to reduce right side power
97
98  // Deceleration zone parameters (keep these good improvements
       )
99  const float DECELERATION_ZONE = 0.5;     // Start slowing
       down 0.5m before target
100 const int MIN_DECELERATION_SPEED = 80;   // Minimum speed
       during deceleration
101
102 // CORRECTED: Realistic turn completion parameters (tightened
        for accuracy)
103 const float TURN_TOLERANCE = 4.0;        // Reduced from 5.0
       for more accurate turns
104 const unsigned long TURN_TIMEOUT = 15000; // 15 second
       timeout for turns
105
106 // === ENHANCED COMPLEMENTARY FILTER PARAMETERS ===//
107
108 // GPS quality thresholds for adaptive weighting
109 const float EXCELLENT_HDOP = 1.0;        // HDOP < 1.0 =
       excellent
110 const float GOOD_HDOP = 2.0;             // HDOP < 2.0 = good
111 const float POOR_HDOP = 5.0;             // HDOP < 5.0 = poor
112 const int MIN_SATELLITES = 4;            // Minimum
       satellites for GPS use
113 const int GOOD_SATELLITES = 6;           // Good satellite
       count
114 const int EXCELLENT_SATELLITES = 8;      // Excellent
       satellite count
115
116 // Complementary filter parameters
117 const float MAX_GPS_WEIGHT = 0.9;        // Maximum GPS
       weight (excellent conditions)
```

```
118 const float MIN_GPS_WEIGHT = 0.1;          // Minimum GPS
       weight (poor conditions)
119 const float DEFAULT_GPS_WEIGHT = 0.7;      // Default weight
       when GPS quality unknown
120
121 // Speed-based fusion parameters
122 const float STATIONARY_SPEED = 0.1;        // m/s - considered
       stationary
123 const float HIGH_SPEED = 1.5;              // m/s - high speed
       threshold
124 const float SPEED_WEIGHT_FACTOR = 0.2;     // How much speed
       affects weighting
125
126 // Objects for sensor interfaces
127 TinyGPSPlus gps;                           // GPS parser object
128 HardwareSerial gpsSerial(2);               // UART2 for GPS
       communication
129 HardwareSerial SerialLidar(1);             // UART1 for LiDAR
       communication
130 MPU6050 mpu;                               // IMU object
131
132 // === GLOBAL VARIABLES ====//
133
134 // Navigation state variables
135 float lat = 0.0;                           // Current latitude
       (degrees)
136 float lon = 0.0;                           // Current longitude
        (degrees)
137 float heading = 0.0;                       // Current heading (
       radians)
138 float target_heading_straight = 0.0;       // Target heading
       for straight line motion (radians)
139 float vel_north = 0.0;                     // Velocity north
       component (m/s)
140 float vel_east = 0.0;                      // Velocity east
       component (m/s)
141 unsigned long last_time = 0;              // Last update time
       (ms)
142 bool gps_available = false;               // GPS validity flag
143 bool imu_available = false;               // IMU status flag
144
```

```
145 // Enhanced fusion variables
146 float current_speed = 0.0;                // Current speed (m/
    s)
147 float last_gps_weight = DEFAULT_GPS_WEIGHT; // Previous GPS
    weight for smoothing
148 float fusion_quality_score = 0.0;         // Overall fusion
    quality (0-1)
149 unsigned long last_gps_update = 0;        // Last GPS update
    time
150 unsigned long gps_update_interval = 0;    // GPS update rate
151
152 // Heading correction PID variables
153 float heading_integral = 0.0;
154 float heading_prev_error = 0.0;
155 unsigned long last_pid_time = 0;
156
157 // Mission state machine enumeration
158 enum MissionState {
159   INITIALIZING,
160   READY,
161   LEG1_MOVING,
162   LEG1_COMPLETE,
163   TURNING_RIGHT_1,
164   TURN1_COMPLETE,
165   LEG2_MOVING,
166   LEG2_COMPLETE,
167   TURNING_RIGHT_2,
168   TURN2_COMPLETE,
169   LEG3_MOVING,
170   LEG3_COMPLETE,
171   TURNING_RIGHT_3,
172   TURN3_COMPLETE,
173   LEG4_MOVING,
174   OBSTACLE_STOP,
175   MISSION_COMPLETE
176 };
177
178 MissionState current_mission_state = INITIALIZING;
179
180 // Mission tracking variables
181 bool missionActive = false;
```

```
182 unsigned long missionStartTime = 0;
183 float leg1_initial_lat = 0.0, leg1_initial_lon = 0.0;
184 float leg2_initial_lat = 0.0, leg2_initial_lon = 0.0;
185 float leg3_initial_lat = 0.0, leg3_initial_lon = 0.0;
186 float leg4_initial_lat = 0.0, leg4_initial_lon = 0.0;
187 float leg1_distance_traveled = 0.0;
188 float leg2_distance_traveled = 0.0;
189 float leg3_distance_traveled = 0.0;
190 float leg4_distance_traveled = 0.0;
191
192 // Turn tracking variables
193 unsigned long turn_start_time = 0;
194 float turn_initial_heading = 0.0;
195 float turn_target_heading = 0.0;
196 unsigned long last_turn_update = 0;
197
198 // LiDAR obstacle detection variables
199 bool obstacleDetected = false;
200 unsigned long last_lidar_scan = 0;
201 const unsigned long LIDAR_SCAN_INTERVAL = 100;
202 int consecutive_obstacle_readings = 0;
203 int consecutive_clear_readings = 0;
204
205 // Statistics tracking
206 struct MissionStatistics {
207   unsigned long gps_updates = 0;
208   unsigned long imu_updates = 0;
209   unsigned long gps_fusion_count = 0;
210   unsigned long imu_only_count = 0;
211   unsigned long valid_lidar = 0;
212   unsigned long invalid_lidar = 0;
213   unsigned long obstacles_detected = 0;
214   unsigned long pid_corrections = 0;
215 } mission_stats;
216
217 // === SETUP FUNCTION ====//
218
219 void setup() {
220   Serial.begin(115200);
221   delay(1000);
222
```

```
223   Serial.println("\n==ENHANCED AUTONOMOUS NAVIGATION SYSTEM
          ==");
224   Serial.println("Version 5.7 - GPS/IMU Adaptive Fusion +
          LiDAR Obstacle Detection");
225   Serial.println("ESP32-S3 | NEO-M8N GPS | MPU-6050 IMU |
          RPLIDAR A1M8");
226   Serial.println("====================================\n");
227
228   // Initialize I2C for MPU6050
229   Wire.begin(SDA_PIN, SCL_PIN);
230   Wire.setClock(400000);
231   delay(100);
232
233   // Initialize IMU
234   Serial.print("Initializing MPU6050 IMU... ");
235   mpu.initialize();
236
237   if (mpu.testConnection()) {
238     Serial.println("SUCCESS");
239     imu_available = true;
240
241     // Configure IMU settings for better accuracy
242     mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);
243     mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
244     mpu.setDLPFMode(MPU6050_DLPF_BW_42);
245
246     Serial.println("IMU configured: 250  /s gyro, 2g accel,
          42Hz filter");
247   } else {
248     Serial.println("FAILED - System will use GPS-only
          navigation");
249     imu_available = false;
250   }
251
252   // Initialize GPS
253   Serial.print("Initializing NEO-M8N GPS... ");
254   gpsSerial.begin(GPS_BAUD, SERIAL_8N1, GPS_RX, GPS_TX);
255   delay(500);
256   Serial.println("UART2 initialized at 9600 baud");
257
258   // Initialize LoRa
```

```
259   Serial.print("Initializing LoRa Ra-02... ");
260   SPI.begin(LORA_SCK, LORA_MISO, LORA_MOSI, LORA_SS);
261   LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
262
263   if (!LoRa.begin(433E6)) {
264     Serial.println("FAILED - Telemetry disabled");
265   } else {
266     LoRa.setSpreadingFactor(12);
267     LoRa.setSignalBandwidth(125E3);
268     LoRa.setCodingRate4(8);
269     LoRa.setTxPower(20);
270     Serial.println("SUCCESS (433MHz, SF12, BW125, CR4/8, 20
          dBm)");
271   }
272
273   // Initialize LiDAR
274   Serial.print("Initializing RPLIDAR A1M8... ");
275   SerialLidar.begin(115200, SERIAL_8N1, RPLIDAR_RX,
        RPLIDAR_TX);
276   pinMode(RPLIDAR_MOTOR, OUTPUT);
277   analogWrite(RPLIDAR_MOTOR, LIDAR_MOTOR_PWM);
278   delay(1000);
279   Serial.println("UART1 initialized at 115200 baud, motor
        started");
280
281   // Initialize motor control pins
282   Serial.print("Initializing motor controllers... ");
283   pinMode(FL_IN1, OUTPUT); pinMode(FL_IN2, OUTPUT); pinMode(
        EN_FL, OUTPUT);
284   pinMode(FR_IN1, OUTPUT); pinMode(FR_IN2, OUTPUT); pinMode(
        EN_FR, OUTPUT);
285   pinMode(RL_IN1, OUTPUT); pinMode(RL_IN2, OUTPUT); pinMode(
        EN_RL, OUTPUT);
286   pinMode(RR_IN1, OUTPUT); pinMode(RR_IN2, OUTPUT); pinMode(
        EN_RR, OUTPUT);
287
288   stopAllMotors();
289   Serial.println("SUCCESS - All motors initialized and
        stopped");
290
291   // Wait for GPS fix with enhanced feedback
```

```
292   Serial.println("\n--- Waiting for GPS fix ---");
293   Serial.println("Requirements: Valid position + 4+
         satellites + HDOP < 5.0");
294
295   unsigned long gps_wait_start = millis();
296   int last_sat_count = 0;
297   float last_hdop = 99.9;
298
299   while (!gps.location.isValid() || gps.satellites.value() <
         MIN_SATELLITES ||
300           gps.hdop.hdop() > POOR_HDOP) {
301
302     while (gpsSerial.available() > 0) {
303       gps.encode(gpsSerial.read());
304     }
305
306     // Print status update every 3 seconds
307     if (millis() - gps_wait_start > 3000) {
308       int current_sats = gps.satellites.isValid() ? gps.
             satellites.value() : 0;
309       float current_hdop = gps.hdop.isValid() ? gps.hdop.hdop
             () : 99.9;
310
311       if (current_sats != last_sat_count || abs(current_hdop
             - last_hdop) > 0.5) {
312         Serial.printf("GPS Status: Sats=%d, HDOP=%.1f, Fix=%s
               \n",
313                       current_sats, current_hdop,
314                       gps.location.isValid() ? "YES" : "NO");
315         last_sat_count = current_sats;
316         last_hdop = current_hdop;
317       }
318
319       gps_wait_start = millis();
320     }
321
322     delay(100);
323   }
324
325   // Initialize position and heading
326   lat = gps.location.lat();
```

```
327   lon = gps.location.lng();
328   gps_available = true;
329
330   if (imu_available) {
331     int16_t ax, ay, az, gx, gy, gz;
332     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
333     heading = atan2(ay, ax);
334   } else {
335     heading = 0.0;
336   }
337
338   target_heading_straight = heading;
339   last_time = millis();
340   last_pid_time = millis();
341
342   // Print initial fusion status
343   Serial.println("\n========== INITIAL SYSTEM STATUS
         ==========");
344   Serial.printf("GPS: Lat=%.6f, Lon=%.6f\n", lat, lon);
345   Serial.printf("Satellites: %d, HDOP: %.1f\n", gps.
         satellites.value(), gps.hdop.hdop());
346   Serial.printf("Initial Heading: %.1f  \n", heading * 180.0
         / PI);
347   Serial.printf("GPS Weight: %.2f, IMU Weight: %.2f\n",
348                 DEFAULT_GPS_WEIGHT, 1.0 - DEFAULT_GPS_WEIGHT)
                      ;
349   Serial.println("=========================================\
         n");
350
351   current_mission_state = READY;
352   Serial.println("System READY. Send 'START' command via
         Serial or LoRa to begin mission.\n");
353 }
354
355 // ===== MAIN LOOP =====//
356
357 void loop() {
358   unsigned long current_time = millis();
359
360   // Update sensors
361   updateGPS();
```

```
362   updateIMU();
363
364   // Calculate adaptive GPS weight based on quality and
        conditions
365   float gps_weight = calculateAdaptiveGPSWeight();
366
367   // Update position and heading with enhanced fusion
368   updatePositionWithEnhancedFusion(gps_weight);
369
370   // Check for mission start command
371   if (current_mission_state == READY) {
372     if (Serial.available() > 0) {
373       String command = Serial.readStringUntil('\n');
374       command.trim();
375       command.toUpperCase();
376
377       if (command == "START") {
378         startMission();
379       }
380     }
381
382     // Check LoRa for start command
383     int packetSize = LoRa.parsePacket();
384     if (packetSize > 0) {
385       String loraCommand = "";
386       while (LoRa.available()) {
387         loraCommand += (char)LoRa.read();
388       }
389       loraCommand.trim();
390       loraCommand.toUpperCase();
391
392       if (loraCommand == "START") {
393         startMission();
394       }
395     }
396   }
397
398   // Scan LiDAR for obstacles (only during movement, not
        turns)
399   if (current_time - last_lidar_scan >= LIDAR_SCAN_INTERVAL)
        {
```

```
400     if (current_mission_state != TURNING_RIGHT_1 &&
401         current_mission_state != TURNING_RIGHT_2 &&
402         current_mission_state != TURNING_RIGHT_3 &&
403         current_mission_state != READY &&
404         current_mission_state != INITIALIZING &&
405         current_mission_state != OBSTACLE_STOP &&
406         current_mission_state != MISSION_COMPLETE) {
407
408       scanLidar();
409     }
410     last_lidar_scan = current_time;
411   }
412
413   // Execute mission state machine
414   if (missionActive) {
415     executeMissionStateMachine();
416   }
417
418   // Print status every 2 seconds
419   static unsigned long last_status_print = 0;
420   if (current_time - last_status_print >= 2000) {
421     printMissionStatus();
422     last_status_print = current_time;
423   }
424
425   // Emergency stop if IMU fails during mission
426   if (!imu_available && missionActive) {
427     stopAllMotors();
428     current_mission_state = OBSTACLE_STOP;
429     logMessageLoRa("EMERGENCY: IMU failure - Mission halted")
        ;
430     Serial.println("EMERGENCY: IMU failure detected - Mission
        halted");
431   }
432
433   delay(1);
434 }
435
436 // ==== SENSOR UPDATE FUNCTIONS =====//
437
438 void updateGPS() {
```

```
439   while (gpsSerial.available() > 0) {
440     char c = gpsSerial.read();
441     if (gps.encode(c)) {
442       if (gps.location.isValid() && gps.satellites.value() >=
              MIN_SATELLITES) {
443         gps_available = true;
444         mission_stats.gps_updates++;
445
446         // Update GPS timing for fusion calculations
447         unsigned long current_time = millis();
448         if (last_gps_update > 0) {
449           gps_update_interval = current_time -
                 last_gps_update;
450         }
451         last_gps_update = current_time;
452       } else {
453         gps_available = false;
454       }
455     }
456   }
457 }
458
459 void updateIMU() {
460   if (!imu_available) return;
461
462   mission_stats.imu_updates++;
463   // IMU data is read in updatePositionWithEnhancedFusion()
464 }
465
466 // ===== ENHANCED FUSION FUNCTIONS ======//
467
468 float calculateAdaptiveGPSWeight() {
469   if (!gps_available || !gps.hdop.isValid()) {
470     return 0.0;  // No GPS available, use IMU only
471   }
472
473   float hdop = gps.hdop.hdop();
474   int sat_count = gps.satellites.value();
475   float base_weight = DEFAULT_GPS_WEIGHT;
476
477   // Calculate HDOP-based weight
```

```
478   if (hdop < EXCELLENT_HDOP) {
479     base_weight = MAX_GPS_WEIGHT;
480   } else if (hdop < GOOD_HDOP) {
481     base_weight = 0.8;
482   } else if (hdop < POOR_HDOP) {
483     base_weight = 0.5;
484   } else {
485     base_weight = MIN_GPS_WEIGHT;
486   }
487
488   // Adjust based on satellite count
489   if (sat_count >= EXCELLENT_SATELLITES) {
490     base_weight = min(base_weight + 0.1, MAX_GPS_WEIGHT);
491   } else if (sat_count < GOOD_SATELLITES) {
492     base_weight = max(base_weight - 0.1, MIN_GPS_WEIGHT);
493   }
494
495   // Speed-based adjustment
496   if (gps.speed.isValid()) {
497     current_speed = gps.speed.mps();
498
499     if (current_speed < STATIONARY_SPEED) {
500       // Stationary - reduce GPS weight (GPS drift is more
          noticeable)
501       base_weight = max(base_weight - 0.2, MIN_GPS_WEIGHT);
502     } else if (current_speed > HIGH_SPEED) {
503       // High speed - increase GPS weight (GPS is more
          reliable)
504       base_weight = min(base_weight + 0.1, MAX_GPS_WEIGHT);
505     }
506   }
507
508   // Smooth weight transitions
509   float smoothed_weight = 0.7 * base_weight + 0.3 *
       last_gps_weight;
510   last_gps_weight = smoothed_weight;
511
512   // Calculate fusion quality score
513   fusion_quality_score = smoothed_weight * (1.0 - (hdop /
       POOR_HDOP));
514
```

```
515   return smoothed_weight;
516 }
517
518 void updatePositionWithEnhancedFusion(float gps_weight) {
519   unsigned long current_time = millis();
520   float dt = (current_time - last_time) / 1000.0;
521
522   if (dt <= 0.0 || dt > 1.0) {
523     last_time = current_time;
524     return;
525   }
526
527   float imu_weight = 1.0 - gps_weight;
528
529   // Get IMU data
530   int16_t ax, ay, az, gx, gy, gz;
531   float imu_heading = heading;
532
533   if (imu_available) {
534     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
535
536     // Calculate heading from gyroscope (yaw rate integration
        )
537     float gyro_z = gz / 131.0;  // Convert to degrees/sec (
        FS_SEL=0)
538     imu_heading = heading + (gyro_z * PI / 180.0) * dt;
539
540     // Normalize heading
541     while (imu_heading > PI) imu_heading -= 2.0 * PI;
542     while (imu_heading < -PI) imu_heading += 2.0 * PI;
543   }
544
545   // Fuse heading
546   if (gps_weight > 0.0 && gps_available && gps.course.isValid
      ()) {
547     float gps_heading = gps.course.deg() * PI / 180.0;
548
549     // Handle 360   wrap-around
550     float heading_diff = gps_heading - imu_heading;
551     if (heading_diff > PI) heading_diff -= 2.0 * PI;
552     if (heading_diff < -PI) heading_diff += 2.0 * PI;
```

```
553
554     heading = imu_heading + gps_weight * heading_diff;
555     mission_stats.gps_fusion_count++;
556   } else {
557     heading = imu_heading;
558     mission_stats.imu_only_count++;
559   }
560
561   // Normalize final heading
562   while (heading > PI) heading -= 2.0 * PI;
563   while (heading < -PI) heading += 2.0 * PI;
564
565   // Fuse position
566   if (gps_weight > 0.0 && gps_available) {
567     float gps_lat = gps.location.lat();
568     float gps_lon = gps.location.lng();
569
570     // IMU-based dead reckoning position estimate
571     float imu_dlat = vel_north * dt / R_EARTH;
572     float imu_dlon = vel_east * dt / (R_EARTH * cos(lat * PI
          / 180.0));
573     float imu_lat = lat + imu_dlat * 180.0 / PI;
574     float imu_lon = lon + imu_dlon * 180.0 / PI;
575
576     // Fuse positions
577     lat = imu_lat + gps_weight * (gps_lat - imu_lat);
578     lon = imu_lon + gps_weight * (gps_lon - imu_lon);
579
580     // Update velocity from GPS
581     if (gps.speed.isValid()) {
582       vel_north = gps.speed.mps() * cos(heading);
583       vel_east = gps.speed.mps() * sin(heading);
584     }
585   } else {
586     // IMU-only dead reckoning
587     float dlat = vel_north * dt / R_EARTH;
588     float dlon = vel_east * dt / (R_EARTH * cos(lat * PI /
          180.0));
589
590     lat += dlat * 180.0 / PI;
591     lon += dlon * 180.0 / PI;
```

```
592   }
593
594   last_time = current_time;
595 }
596
597 // ===== MISSION CONTROL FUNCTIONS =====//
598
599 void startMission() {
600   Serial.println("\n========== MISSION START ==========");
601   Serial.println("Rectangular path with enhanced GPS/IMU
        fusion");
602   Serial.println("4m x 3m rectangle with 90   turns");
603   Serial.println("==================================\n");
604
605   missionActive = true;
606   missionStartTime = millis();
607
608   leg1_initial_lat = lat;
609   leg1_initial_lon = lon;
610   leg1_distance_traveled = 0.0;
611
612   current_mission_state = LEG1_MOVING;
613   target_heading_straight = heading;
614
615   // Reset PID
616   heading_integral = 0.0;
617   heading_prev_error = 0.0;
618   last_pid_time = millis();
619
620   // Reset obstacle detection
621   obstacleDetected = false;
622   consecutive_obstacle_readings = 0;
623   consecutive_clear_readings = 0;
624
625   // Reset statistics
626   mission_stats = MissionStatistics();
627
628   logMessageLoRa("MISSION_START");
629   Serial.println("LEG 1: Moving forward with enhanced fusion
        ");
630 }
```

```
631
632 void executeMissionStateMachine() {
633   switch (current_mission_state) {
634     case LEG1_MOVING:
635       executeLeg1();
636       break;
637
638     case TURNING_RIGHT_1:
639       executeTurn(1);
640       break;
641
642     case LEG2_MOVING:
643       executeLeg2();
644       break;
645
646     case TURNING_RIGHT_2:
647       executeTurn(2);
648       break;
649
650     case LEG3_MOVING:
651       executeLeg3();
652       break;
653
654     case TURNING_RIGHT_3:
655       executeTurn(3);
656       break;
657
658     case LEG4_MOVING:
659       executeLeg4();
660       break;
661
662     case OBSTACLE_STOP:
663       handleObstacleStop();
664       break;
665
666     case MISSION_COMPLETE:
667       handleMissionComplete();
668       break;
669
670     default:
671       break;
```

```
672   }
673 }
674
675 // ===== LEG EXECUTION FUNCTIONS =====//
676
677 void executeLeg1() {
678   if (obstacleDetected) {
679     stopAllMotors();
680     current_mission_state = OBSTACLE_STOP;
681     logMessageLoRa("LEG1_OBSTACLE_STOP");
682     return;
683   }
684
685   leg1_distance_traveled = calculateDistance(leg1_initial_lat
        , leg1_initial_lon, lat, lon);
686
687   if (leg1_distance_traveled >= LEG1_TARGET_DISTANCE) {
688     stopAllMotors();
689     current_mission_state = LEG1_COMPLETE;
690
691     delay(500);
692     current_mission_state = TURNING_RIGHT_1;
693     initiateTurn();
694     logMessageLoRa("LEG1_COMPLETE_TURN1_START");
695   } else {
696     moveForwardWithHeadingControl(leg1_distance_traveled,
          LEG1_TARGET_DISTANCE);
697   }
698 }
699
700 void executeLeg2() {
701   if (obstacleDetected) {
702     stopAllMotors();
703     current_mission_state = OBSTACLE_STOP;
704     logMessageLoRa("LEG2_OBSTACLE_STOP");
705     return;
706   }
707
708   leg2_distance_traveled = calculateDistance(leg2_initial_lat
        , leg2_initial_lon, lat, lon);
709
```

```
710   if (leg2_distance_traveled >= LEG2_TARGET_DISTANCE) {
711     stopAllMotors();
712     current_mission_state = LEG2_COMPLETE;
713
714     delay(500);
715     current_mission_state = TURNING_RIGHT_2;
716     initiateTurn();
717     logMessageLoRa("LEG2_COMPLETE_TURN2_START");
718   } else {
719     moveForwardWithHeadingControl(leg2_distance_traveled,
          LEG2_TARGET_DISTANCE);
720   }
721 }
722
723 void executeLeg3() {
724   if (obstacleDetected) {
725     stopAllMotors();
726     current_mission_state = OBSTACLE_STOP;
727     logMessageLoRa("LEG3_OBSTACLE_STOP");
728     return;
729   }
730
731   leg3_distance_traveled = calculateDistance(leg3_initial_lat
       , leg3_initial_lon, lat, lon);
732
733   if (leg3_distance_traveled >= LEG3_TARGET_DISTANCE) {
734     stopAllMotors();
735     current_mission_state = LEG3_COMPLETE;
736
737     delay(500);
738     current_mission_state = TURNING_RIGHT_3;
739     initiateTurn();
740     logMessageLoRa("LEG3_COMPLETE_TURN3_START");
741   } else {
742     moveForwardWithHeadingControl(leg3_distance_traveled,
          LEG3_TARGET_DISTANCE);
743   }
744 }
745
746 void executeLeg4() {
747   if (obstacleDetected) {
```

```
748     stopAllMotors();
749     current_mission_state = OBSTACLE_STOP;
750     logMessageLoRa("LEG4_OBSTACLE_STOP");
751     return;
752   }
753
754   leg4_distance_traveled = calculateDistance(leg4_initial_lat
        , leg4_initial_lon, lat, lon);
755
756   if (leg4_distance_traveled >= LEG4_TARGET_DISTANCE) {
757     stopAllMotors();
758     current_mission_state = MISSION_COMPLETE;
759     logMessageLoRa("MISSION_COMPLETE");
760   } else {
761     moveForwardWithHeadingControl(leg4_distance_traveled,
          LEG4_TARGET_DISTANCE);
762   }
763 }
764
765 // ==== TURN EXECUTION FUNCTIONS ====//
766
767 void initiateTurn() {
768   turn_start_time = millis();
769   turn_initial_heading = heading;
770   turn_target_heading = turn_initial_heading + (TURN_ANGLE *
        PI / 180.0);
771
772   // Normalize target heading
773   while (turn_target_heading > PI) turn_target_heading -= 2.0
        * PI;
774   while (turn_target_heading < -PI) turn_target_heading +=
        2.0 * PI;
775
776   last_turn_update = millis();
777 }
778
779 void executeTurn(int turn_number) {
780   unsigned long current_time = millis();
781
782   // Check timeout
783   if (current_time - turn_start_time > TURN_TIMEOUT) {
```

```
784     stopAllMotors();
785     Serial.println("TURN TIMEOUT - Proceeding to next leg");
786     completeTurn(turn_number);
787     return;
788   }
789
790   // Calculate heading error
791   float current_heading_deg = heading * 180.0 / PI;
792   float target_heading_deg = turn_target_heading * 180.0 / PI
        ;
793   float heading_error = target_heading_deg -
        current_heading_deg;
794
795   // Normalize error to [-180, 180]
796   while (heading_error > 180.0) heading_error -= 360.0;
797   while (heading_error < -180.0) heading_error += 360.0;
798
799   // Check if turn is complete
800   if (abs(heading_error) < TURN_TOLERANCE) {
801     stopAllMotors();
802     delay(500);
803     completeTurn(turn_number);
804     return;
805   }
806
807   // Execute turn
808   turnRight(TURN_SPEED);
809
810   // Log progress every 500ms
811   if (current_time - last_turn_update >= 500) {
812     Serial.printf("Turn %d: Error=%.1f  , Current=%.1f  ,
          Target=%.1f  \n",
813                   turn_number, heading_error,
                        current_heading_deg, target_heading_deg)
                        ;
814     last_turn_update = current_time;
815   }
816 }
817
818 void completeTurn(int turn_number) {
819   target_heading_straight = heading;
```

```
820
821   // Reset PID for next leg
822   heading_integral = 0.0;
823   heading_prev_error = 0.0;
824   last_pid_time = millis();
825
826   switch (turn_number) {
827     case 1:
828       current_mission_state = TURN1_COMPLETE;
829       leg2_initial_lat = lat;
830       leg2_initial_lon = lon;
831       leg2_distance_traveled = 0.0;
832       delay(500);
833       current_mission_state = LEG2_MOVING;
834       Serial.println("LEG 2: Moving forward");
835       break;
836
837     case 2:
838       current_mission_state = TURN2_COMPLETE;
839       leg3_initial_lat = lat;
840       leg3_initial_lon = lon;
841       leg3_distance_traveled = 0.0;
842       delay(500);
843       current_mission_state = LEG3_MOVING;
844       Serial.println("LEG 3: Moving forward");
845       break;
846
847     case 3:
848       current_mission_state = TURN3_COMPLETE;
849       leg4_initial_lat = lat;
850       leg4_initial_lon = lon;
851       leg4_distance_traveled = 0.0;
852       delay(500);
853       current_mission_state = LEG4_MOVING;
854       Serial.println("LEG 4: Final leg - returning to start")
          ;
855       break;
856   }
857 }
858
859 // ==== MOVEMENT CONTROL FUNCTIONS ====//
```

```
860
861 void moveForwardWithHeadingControl(float distance_traveled,
       float target_distance) {
862   float current_heading_deg = heading * 180.0 / PI;
863   float target_heading_deg = target_heading_straight * 180.0
        / PI;
864   float heading_error = target_heading_deg -
        current_heading_deg;
865
866   // Normalize error to [-180, 180]
867   while (heading_error > 180.0) heading_error -= 360.0;
868   while (heading_error < -180.0) heading_error += 360.0;
869
870   // Apply deadband
871   if (abs(heading_error) < HEADING_DEADBAND) {
872     heading_error = 0.0;
873   }
874
875   // Calculate PID correction
876   unsigned long current_time = millis();
877   float dt = (current_time - last_pid_time) / 1000.0;
878
879   if (dt > 0.0 && dt < 1.0) {
880     float p_term = HEADING_KP * heading_error;
881     heading_integral += heading_error * dt;
882     heading_integral = constrain(heading_integral, -20.0,
          20.0);
883     float i_term = HEADING_KI * heading_integral;
884     float d_term = HEADING_KD * (heading_error -
          heading_prev_error) / dt;
885
886     float pid_correction = p_term + i_term + d_term;
887     pid_correction = constrain(pid_correction, -
          MAX_HEADING_CORRECTION, MAX_HEADING_CORRECTION);
888
889     heading_prev_error = heading_error;
890     last_pid_time = current_time;
891
892     if (abs(pid_correction) > 1.0) {
893       mission_stats.pid_corrections++;
894     }
```

```
895
896     // Calculate speeds with deceleration zone
897     int base_speed = BASE_SPEED;
898     float distance_remaining = target_distance -
           distance_traveled;
899
900     if (distance_remaining < DECELERATION_ZONE) {
901       float decel_factor = distance_remaining /
             DECELERATION_ZONE;
902       base_speed = MIN_DECELERATION_SPEED +
903                    (BASE_SPEED - MIN_DECELERATION_SPEED) *
                        decel_factor;
904     }
905
906     // Apply motor corrections
907     int left_speed = base_speed - pid_correction;
908     int right_speed = base_speed + pid_correction;
909
910     left_speed = constrain(left_speed, 0, 255);
911     right_speed = constrain(right_speed, 0, 255);
912
913     setMotorSpeeds(left_speed, right_speed, left_speed,
           right_speed);
914   }
915 }
916
917 void setMotorSpeeds(int fl_speed, int fr_speed, int rl_speed,
       int rr_speed) {
918   // Apply calibration factors
919   fl_speed = constrain((int)(fl_speed * FL_MOTOR_FACTOR), 0,
         255);
920   fr_speed = constrain((int)(fr_speed * FR_MOTOR_FACTOR), 0,
         255);
921   rl_speed = constrain((int)(rl_speed * RL_MOTOR_FACTOR), 0,
         255);
922   rr_speed = constrain((int)(rr_speed * RR_MOTOR_FACTOR), 0,
         255);
923
924   // Front Left motor
925   digitalWrite(FL_IN1, HIGH);
926   digitalWrite(FL_IN2, LOW);
```

```
927   analogWrite(EN_FL, fl_speed);
928
929   // Front Right motor
930   digitalWrite(FR_IN1, HIGH);
931   digitalWrite(FR_IN2, LOW);
932   analogWrite(EN_FR, fr_speed);
933
934   // Rear Left motor
935   digitalWrite(RL_IN1, HIGH);
936   digitalWrite(RL_IN2, LOW);
937   analogWrite(EN_RL, rl_speed);
938
939   // Rear Right motor
940   digitalWrite(RR_IN1, HIGH);
941   digitalWrite(RR_IN2, LOW);
942   analogWrite(EN_RR, rr_speed);
943 }
944
945 void turnRight(int turn_speed) {
946   // Left motors forward
947   digitalWrite(FL_IN1, HIGH);
948   digitalWrite(FL_IN2, LOW);
949   analogWrite(EN_FL, turn_speed);
950
951   digitalWrite(RL_IN1, HIGH);
952   digitalWrite(RL_IN2, LOW);
953   analogWrite(EN_RL, turn_speed);
954
955   // Right motors backward
956   digitalWrite(FR_IN1, LOW);
957   digitalWrite(FR_IN2, HIGH);
958   analogWrite(EN_FR, turn_speed);
959
960   digitalWrite(RR_IN1, LOW);
961   digitalWrite(RR_IN2, HIGH);
962   analogWrite(EN_RR, turn_speed);
963 }
964
965 void stopAllMotors() {
966   digitalWrite(FL_IN1, LOW);
967   digitalWrite(FL_IN2, LOW);
```

```
968   analogWrite(EN_FL, 0);
969
970   digitalWrite(FR_IN1, LOW);
971   digitalWrite(FR_IN2, LOW);
972   analogWrite(EN_FR, 0);
973
974   digitalWrite(RL_IN1, LOW);
975   digitalWrite(RL_IN2, LOW);
976   analogWrite(EN_RL, 0);
977
978   digitalWrite(RR_IN1, LOW);
979   digitalWrite(RR_IN2, LOW);
980   analogWrite(EN_RR, 0);
981 }
982
983 // ===== LIDAR FUNCTIONS =====//
984
985 void scanLidar() {
986   int obstacle_count = 0;
987   int valid_readings = 0;
988
989   while (SerialLidar.available() > 0) {
990     uint8_t byte = SerialLidar.read();
991
992     // Simple packet detection (RPLIDAR A1M8 protocol)
993     static uint8_t packet[5];
994     static int packet_index = 0;
995
996     packet[packet_index++] = byte;
997
998     if (packet_index >= 5) {
999       packet_index = 0;
1000
1001      // Parse distance and angle
1002      uint16_t distance = (packet[3] << 8) | packet[2];
1003      float angle = ((packet[1] << 8) | packet[0]) / 64.0;
1004      uint8_t quality = packet[4] >> 2;
1005
1006      // Check if reading is in FOV and valid
1007      bool in_fov = false;
1008      if (FOV_START_ANGLE > FOV_END_ANGLE) {
```

```
1009        in_fov = (angle >= FOV_START_ANGLE || angle <=
                FOV_END_ANGLE);
1010      } else {
1011        in_fov = (angle >= FOV_START_ANGLE && angle <=
                FOV_END_ANGLE);
1012      }
1013
1014      if (in_fov && quality >= MIN_QUALITY &&
1015          distance >= MIN_OBSTACLE_DISTANCE &&
1016          distance <= MAX_REALISTIC_DISTANCE) {
1017
1018        valid_readings++;
1019        mission_stats.valid_lidar++;
1020
1021        if (distance <= OBSTACLE_THRESHOLD) {
1022          obstacle_count++;
1023        }
1024      } else if (in_fov) {
1025        mission_stats.invalid_lidar++;
1026      }
1027    }
1028  }
1029
1030  // Update obstacle detection with hysteresis
1031  if (obstacle_count >= MIN_OBSTACLE_READINGS) {
1032    consecutive_obstacle_readings++;
1033    consecutive_clear_readings = 0;
1034
1035    if (consecutive_obstacle_readings >= 2 && !
           obstacleDetected) {
1036      obstacleDetected = true;
1037      mission_stats.obstacles_detected++;
1038      Serial.printf("OBSTACLE DETECTED! Count=%d, Valid=%d\n
             ",
1039                    obstacle_count, valid_readings);
1040    }
1041  } else if (valid_readings >= MIN_CLEAR_READINGS) {
1042    consecutive_clear_readings++;
1043    consecutive_obstacle_readings = 0;
1044
```

```
1045      if (consecutive_clear_readings >= 2 && obstacleDetected)
             {
1046        obstacleDetected = false;
1047        Serial.println("Path CLEAR - Obstacle removed");
1048      }
1049    }
1050  }
1051
1052  // ===== UTILITY FUNCTIONS =====//
1053
1054  float calculateDistance(float lat1, float lon1, float lat2,
       float lon2) {
1055    float dlat = (lat2 - lat1) * PI / 180.0;
1056    float dlon = (lon2 - lon1) * PI / 180.0;
1057
1058    float a = sin(dlat / 2.0) * sin(dlat / 2.0) +
1059              cos(lat1 * PI / 180.0) * cos(lat2 * PI / 180.0) *
1060              sin(dlon / 2.0) * sin(dlon / 2.0);
1061
1062    float c = 2.0 * atan2(sqrt(a), sqrt(1.0 - a));
1063    return R_EARTH * c;
1064  }
1065
1066  float calculateMissionProgress() {
1067    float total_distance = LEG1_TARGET_DISTANCE +
         LEG2_TARGET_DISTANCE +
1068                           LEG3_TARGET_DISTANCE +
                                LEG4_TARGET_DISTANCE;
1069    float completed_distance = leg1_distance_traveled +
         leg2_distance_traveled +
1070                               leg3_distance_traveled +
                                    leg4_distance_traveled;
1071    return (completed_distance / total_distance) * 100.0;
1072  }
1073
1074  void logMessageLoRa(String message) {
1075    LoRa.beginPacket();
1076    LoRa.print(message);
1077    LoRa.endPacket();
1078  }
1079
```

```
1080 void handleObstacleStop() {
1081   static unsigned long last_check = 0;
1082   if (millis() - last_check > 1000) {
1083     Serial.println("MISSION HALTED - Obstacle detected.
         Waiting for path to clear...");
1084     last_check = millis();
1085   }
1086
1087   // Check if obstacle is cleared
1088   if (!obstacleDetected) {
1089     Serial.println("Path cleared - Resuming mission");
1090
1091     // Resume appropriate leg
1092     if (leg1_distance_traveled < LEG1_TARGET_DISTANCE) {
1093       current_mission_state = LEG1_MOVING;
1094     } else if (leg2_distance_traveled < LEG2_TARGET_DISTANCE)
         {
1095       current_mission_state = LEG2_MOVING;
1096     } else if (leg3_distance_traveled < LEG3_TARGET_DISTANCE)
         {
1097       current_mission_state = LEG3_MOVING;
1098     } else if (leg4_distance_traveled < LEG4_TARGET_DISTANCE)
         {
1099       current_mission_state = LEG4_MOVING;
1100     }
1101   }
1102 }
1103
1104 void handleMissionComplete() {
1105   static bool completion_logged = false;
1106
1107   if (!completion_logged) {
1108     float return_distance = calculateDistance(
         leg1_initial_lat, leg1_initial_lon, lat, lon);
1109
1110     Serial.println("\n===MISSION COMPLETE ===");
1111     Serial.printf("Total mission time: %.1f seconds\n",
1112                   (millis() - missionStartTime) / 1000.0);
1113     Serial.printf("Return accuracy: %.2f meters from start\n
         ", return_distance);
1114     Serial.println("================================\n");
```

```
1115
1116      logMessageLoRa("MISSION_COMPLETE");
1117      completion_logged = true;
1118      missionActive = false;
1119    }
1120  }
1121
1122  // ===== STATUS FUNCTIONS =====//
1123
1124  void printMissionStatus() {
1125    Serial.println("\n==ENHANCED NAVIGATION MISSION STATUS ==")
          ;
1126    Serial.print("State: ");
1127
1128    switch (current_mission_state) {
1129      case INITIALIZING: Serial.println("INITIALIZING"); break;
1130      case READY: Serial.println("READY"); break;
1131      case LEG1_MOVING: Serial.println("LEG 1 - MOVING WITH
            ENHANCED FUSION"); break;
1132      case LEG1_COMPLETE: Serial.println("LEG 1 - COMPLETED");
            break;
1133      case TURNING_RIGHT_1: Serial.println("TURNING RIGHT #1 (
            CORRECTED LOGIC)"); break;
1134      case TURN1_COMPLETE: Serial.println("TURN 1 - COMPLETED")
            ; break;
1135      case LEG2_MOVING: Serial.println("LEG 2 - MOVING WITH
            ENHANCED FUSION"); break;
1136      case LEG2_COMPLETE: Serial.println("LEG 2 - COMPLETED");
            break;
1137      case TURNING_RIGHT_2: Serial.println("TURNING RIGHT #2 (
            CORRECTED LOGIC)"); break;
1138      case TURN2_COMPLETE: Serial.println("TURN 2 - COMPLETED")
            ; break;
1139      case LEG3_MOVING: Serial.println("LEG 3 - MOVING WITH
            ENHANCED FUSION"); break;
1140      case LEG3_COMPLETE: Serial.println("LEG 3 - COMPLETED");
            break;
1141      case TURNING_RIGHT_3: Serial.println("TURNING RIGHT #3 (
            CORRECTED LOGIC)"); break;
1142      case TURN3_COMPLETE: Serial.println("TURN 3 - COMPLETED")
            ; break;
```

```
1143      case LEG4_MOVING: Serial.println("LEG 4 - FINAL LEG WITH
              ENHANCED FUSION"); break;
1144      case OBSTACLE_STOP: Serial.println("STOPPED - OBSTACLE
              DETECTED"); break;
1145      case MISSION_COMPLETE: Serial.println("MISSION COMPLETED
              "); break;
1146    }
1147
1148    Serial.println("--- Enhanced Sensor Fusion Status ---");
1149    Serial.print("GPS Weight: "); Serial.print(last_gps_weight,
            3);
1150    Serial.print(" | IMU Weight: "); Serial.println(1.0 -
            last_gps_weight, 3);
1151    Serial.print("Fusion Quality: "); Serial.print(
            fusion_quality_score, 2);
1152    Serial.print(" | Current Speed: "); Serial.print(
            current_speed, 2);
1153    Serial.println(" m/s");
1154
1155    if (gps.hdop.isValid() && gps.satellites.isValid()) {
1156      Serial.print("GPS Quality - HDOP: "); Serial.print(gps.
              hdop.hdop(), 2);
1157      Serial.print(" | Satellites: "); Serial.println(gps.
              satellites.value());
1158    }
1159
1160    Serial.println("--- Mission Progress ---");
1161    Serial.print("Overall Progress: "); Serial.print(
            calculateMissionProgress(), 1);
1162    Serial.println("%");
1163
1164    Serial.print("Leg 1: "); Serial.print(
            leg1_distance_traveled, 2);
1165    Serial.print("m / "); Serial.print(LEG1_TARGET_DISTANCE, 1)
            ; Serial.print("m");
1166    Serial.print(" ("); Serial.print((leg1_distance_traveled /
            LEG1_TARGET_DISTANCE) * 100.0, 1);
1167    Serial.println("%)");
1168
1169    Serial.print("Leg 2: "); Serial.print(
            leg2_distance_traveled, 2);
```

```
1170    Serial.print("m / "); Serial.print(LEG2_TARGET_DISTANCE, 1)
           ; Serial.print("m");
1171    Serial.print(" ("); Serial.print((leg2_distance_traveled /
           LEG2_TARGET_DISTANCE) * 100.0, 1);
1172    Serial.println("%)");
1173
1174    Serial.print("Leg 3: "); Serial.print(
           leg3_distance_traveled, 2);
1175    Serial.print("m / "); Serial.print(LEG3_TARGET_DISTANCE, 1)
           ; Serial.print("m");
1176    Serial.print(" ("); Serial.print((leg3_distance_traveled /
           LEG3_TARGET_DISTANCE) * 100.0, 1);
1177    Serial.println("%)");
1178
1179    Serial.print("Leg 4: "); Serial.print(
           leg4_distance_traveled, 2);
1180    Serial.print("m / "); Serial.print(LEG4_TARGET_DISTANCE, 1)
           ; Serial.print("m");
1181    Serial.print(" ("); Serial.print((leg4_distance_traveled /
           LEG4_TARGET_DISTANCE) * 100.0, 1);
1182    Serial.println("%)");
1183
1184    if (current_mission_state == LEG1_MOVING ||
           current_mission_state == LEG2_MOVING ||
1185        current_mission_state == LEG3_MOVING ||
               current_mission_state == LEG4_MOVING) {
1186
1187      float current_heading_deg = heading * 180.0 / PI;
1188      float target_heading_deg = target_heading_straight *
             180.0 / PI;
1189      float heading_error = target_heading_deg -
             current_heading_deg;
1190
1191      while (heading_error > 180.0) heading_error -= 360.0;
1192      while (heading_error < -180.0) heading_error += 360.0;
1193
1194      Serial.println("--- HEADING CONTROL ---");
1195      Serial.print("Current heading: "); Serial.print(
             current_heading_deg, 1);
1196      Serial.println("  ");
```

```
1197      Serial.print("Target heading: "); Serial.print(
              target_heading_deg, 1);
1198      Serial.println("  ");
1199      Serial.print("Heading error: "); Serial.print(
              heading_error, 1);
1200      Serial.println("  ");
1201    }
1202
1203    Serial.println("--- Position and Navigation ---");
1204    Serial.print("Position: Lat "); Serial.print(lat, 6);
1205    Serial.print(", Lon "); Serial.println(lon, 6);
1206    Serial.print("Current Heading: "); Serial.print(heading *
          180.0 / PI, 1);
1207    Serial.println("  ");
1208
1209    Serial.println("--- System Status ---");
1210    Serial.print("GPS: "); Serial.print(gps_available ? "ACTIVE
          " : "INACTIVE");
1211
1212    if (gps_available) {
1213      Serial.print(" (Sats: "); Serial.print(gps.satellites.
              value());
1214      Serial.print(", HDOP: "); Serial.print(gps.hdop.hdop(),
              1); Serial.print(")");
1215    }
1216
1217    Serial.println();
1218    Serial.print("IMU: "); Serial.print(imu_available ? "ACTIVE
          " : "FAILED");
1219    Serial.println();
1220
1221    Serial.print("ENHANCED FUSION: GPS/IMU weights: ");
1222    Serial.print(last_gps_weight, 2); Serial.print("/");
1223    Serial.println(1.0 - last_gps_weight, 2);
1224
1225    Serial.print("PID Control (Kp="); Serial.print(HEADING_KP,
          1);
1226    Serial.print(", Ki="); Serial.print(HEADING_KI, 1);
1227    Serial.print(", Kd="); Serial.print(HEADING_KD, 1); Serial.
          println(")");
1228
```

```
1229    Serial.print("Motor Cal: FL="); Serial.print(
           FL_MOTOR_FACTOR, 2);
1230    Serial.print(", FR="); Serial.print(FR_MOTOR_FACTOR, 2);
1231    Serial.print(", RL="); Serial.print(RL_MOTOR_FACTOR, 2);
1232    Serial.print(", RR="); Serial.print(RR_MOTOR_FACTOR, 2);
           Serial.println();
1233
1234    Serial.print("LiDAR: Valid readings: "); Serial.print(
           mission_stats.valid_lidar);
1235    Serial.print(", Obstacles detected: "); Serial.println(
           mission_stats.obstacles_detected);
1236
1237    Serial.print("Obstacle Detection: ");
1238
1239    if (current_mission_state == TURNING_RIGHT_1 ||
1240        current_mission_state == TURNING_RIGHT_2 ||
1241        current_mission_state == TURNING_RIGHT_3) {
1242      Serial.println("DISABLED (TURNING)");
1243    } else {
1244      Serial.println(obstacleDetected ? "OBSTACLE PRESENT" : "
           PATH CLEAR");
1245    }
1246
1247    // Enhanced fusion statistics
1248    if (mission_stats.gps_fusion_count > 0 || mission_stats.
           imu_only_count > 0) {
1249      float fusion_ratio = (float)mission_stats.
           gps_fusion_count /
1250                           (mission_stats.gps_fusion_count +
                                mission_stats.imu_only_count) *
                                100.0;
1251
1252      Serial.print("Fusion Stats: GPS operations: "); Serial.
           print(mission_stats.gps_fusion_count);
1253      Serial.print(", IMU-only: "); Serial.print(mission_stats.
           imu_only_count);
1254      Serial.print(" ("); Serial.print(fusion_ratio, 1); Serial
           .println("% GPS available)");
1255    }
1256
1257    if (current_mission_state == MISSION_COMPLETE) {
```

```
1258      float return_distance = calculateDistance(
              leg1_initial_lat, leg1_initial_lon, lat, lon);
1259
1260      Serial.print("FINAL RESULT - Return Accuracy: ");
1261      Serial.print(return_distance, 2); Serial.println(" meters
              from start");
1262
1263      if (return_distance < 1.0) {
1264        Serial.println("    EXCELLENT: Return accuracy <1.0m
              achieved!");
1265      } else if (return_distance < 2.0) {
1266        Serial.println("    GOOD: Return accuracy <2.0m
              achieved");
1267      } else if (return_distance < 3.0) {
1268        Serial.println("    ACCEPTABLE: Return accuracy <3.0m
              achieved");
1269      } else {
1270        Serial.println("    NEEDS IMPROVEMENT: Consider
              parameter tuning");
1271      }
1272
1273      // Final enhanced fusion analysis
1274      Serial.println("\n=== FINAL ENHANCED FUSION ANALYSIS
              ===");
1275      Serial.print("Average GPS weight used: ");
1276
1277      if (mission_stats.gps_fusion_count > 0) {
1278        Serial.println("Dynamic based on quality");
1279      } else {
1280        Serial.println("N/A (IMU-only mission)");
1281      }
1282
1283      Serial.print("Final fusion quality score: "); Serial.
              println(fusion_quality_score, 2);
1284      Serial.println("Enhanced adaptive fusion provided
              improved accuracy");
1285    }
1286
1287    Serial.println("====================================\n");
1288  }
1289
```

```
1290 //**********************End of Code*********************//
```